

Laporan Milestone 2

Syntax Analysis

IF2224 Teori Bahasa Formal dan Automata



Kelompok JEE - SundalnJVM

Billie Bhaskara Wibawa	13524024
Raysha Erviandika Putra	13524050
Muhammad Naufal Romi Annafi	13524058
Dzaki Ahmad Al Hussainy	13524084

PROGRAM STUDI TEKNIK INFORMATIKA

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

JL. GANESA 10, BANDUNG 40132

2026

Daftar Isi

Daftar Isi.....	2
1. Teori Singkat.....	3
1.1. Syntax Analysis (Parser).....	3
1.2. Struktur Pohon.....	3
1.3. Context-Free Grammar.....	4
1.4. Algoritma Recursive Descent Parsing.....	6
1.5. Parse Tree.....	6
2. Perancangan dan Implementasi.....	8
2.1. Teknologi yang digunakan.....	8
2.2. Arsitektur Program.....	8
2.3. Representasi Parse Tree.....	9
2.4. Kelas Parser dan Mekanisme Recursive Descent.....	11
2.5. Pemetaan Grammar ke Fungsi Parser.....	45
2.6. Alur Eksekusi Syntax Analyzer dan Integrasi dengan Lexer.....	46
3. Pengujian.....	48
4. Kesimpulan dan Saran.....	71
4.1. Kesimpulan.....	71
4.2. Saran.....	71
5. Lampiran.....	73
6. Referensi.....	74

1. Teori Singkat

1.1. Syntax Analysis (Parser)

Parsing adalah bagian kedua dari alur kerja compiler setelah bagian lexer. Output dari lexer adalah urutan token yang dihasilkan dari membaca source file. Tugas dari parser ini adalah melakukan penurunan sintaks dari input urutan token berdasarkan aturan model grammar bahasa tertentu. Tujuannya adalah untuk rekognisi sintaks dari bahasa pemrograman Arion apakah valid atau tidak.

Tahapan Parser bertujuan untuk:

1. Memastikan urutan token membentuk struktur sintaks yang valid.
2. Mendeteksi dan melaporkan kesalahan sintaks (syntax error).
3. Mempersiapkan representasi program untuk tahap berikutnya (Semantic Analysis).

1.2. Struktur Pohon

Pohon (*tree*) adalah struktur data non-linear yang digunakan untuk merepresentasikan hubungan hierarkis antar elemen. Struktur pohon mengorganisasikan data dalam bentuk cabang yang mensimulasikan struktur hierarki.

- Akar (Root)
Simpul (*node*) teratas dari pohon yang tidak memiliki induk (*parent*). Seluruh penelusuran struktur pohon selalu diawali dari simpul ini.
- Simpul (Node)
Entitas dasar yang menyimpan informasi atau data. Setiap simpul dapat memiliki simpul turunan.
- Sisi (Edge)
Garis penghubung yang merepresentasikan relasi antara simpul induk dan simpul anak (*child*).
- Daun (Leaf)
Simpul terminal yang terletak di bagian paling bawah dari hierarki, ditandai dengan tidak adanya simpul anak yang terhubung padanya.

- Sub-pohon (Subtree)
Bagian dari pohon yang terdiri dari sebuah simpul beserta seluruh keturunannya.

Dalam konteks CFG, elemen-elemen struktur pohon dipetakan ke dalam entitas tata bahasa formal sebagai berikut:

- Akar (Root)
Selalu merepresentasikan Simpul Awal (*Start Symbol*) dari sebuah CFG,
- Simpul Internal (Internal Nodes)
Seluruh simpul yang bukan daun wajib merupakan simbol Non-terminal (simbol yang masih dapat diturunkan atau diekspansi).
- Daun (Leaf)
Selalu merepresentasikan simbol Terminal (token dasar dari bahasa yang tidak dapat diturunkan lagi) atau string kosong yang dinotasikan dengan ϵ .
- Hasil (Yield)
Pembacaan simpul-simpul daun dari urutan paling kiri ke paling kanan akan menghasilkan *string* input yang valid sesuai dengan bahasa yang didefinisikan oleh CFG tersebut.

1.3. Context-Free Grammar

Aturan tata bahasa ini direpresentasikan dengan context-free Grammar. CFG ini mendefinisikan struktur rekursif Pohon dari suatu bahasa untuk menentukan sintaks dari sebuah bahasa pemrograman. CFG ini memiliki 4 komponen (T,V,S,P) yaitu Terminal, Non-Terminal symbol (Variable), Start State dan Production.

Dalam Milestone 2 ini sudah diberikan CFG yang ditulis dalam Extended Backus-Naur form (EBNF) dalam bahasa pemrograman Arion.

Notasi EBNF akan membacanya seperti ini

1. $\rightarrow$: Definisi (dibaca: "didefinisikan sebagai").
2. $|$: Alternatif (dibaca: "ATAU").

3. [...] atau ? : Opsional (boleh ada atau tidak, maksimal 1 kali). Menggantikan notasi ϵ di CFG.
4. { ... } atau * : Perulangan (boleh muncul 0 kali, 1 kali, atau berkali-kali). Menghilangkan keharusan menggunakan rekursi kiri/kanan.
5. (...) : Pengelompokan aturan agar dievaluasi bersamaan.

Kata yang ditulis dengan kapital adalah variabel, yang ditulis dengan huruf kapitil dan ditebalkan adalah terminal. Berikut hasil production dari bahasa Arion ini

1. PROGRAM $\rightarrow$ PROGRAM-HEADER + DECLARATION-PART + COMPOUND-STATEMENT + **period**
2. PROGRAM-HEADER $\rightarrow$ **programsy** + **ident** + **semicolon**
3. DECLARATION-PART $\rightarrow$ (CONST-DECLARATION)* + (TYPE-DECLARATION)* + (VAR-DECLARATION)* + (SUBPROGRAM-DECLARATION)*
4. CONST-DECLARATION $\rightarrow$ **constsy** + (**ident** + **eql** + CONSTANT + **semicolon**)
5. CONSTANT $\rightarrow$ **charcon** | **string** | [(**plus** | **minus**)? + (**ident** | **intcon** | **realcon**)]
6. TYPE-DECLARATION $\rightarrow$ **typesy** + (**ident** + **eql** + TYPE + **semicolon**) + TYPE-DECLARATION
7. VAR-DECLARATION $\rightarrow$ **varsy** + (IDENTIFIER-LIST + **colon** + TYPE + **semicolon**) + VAR-DECLARATION
8. IDENTIFIER-LIST $\rightarrow$ **ident** (**comma** + **ident**)*
9. TYPE $\rightarrow$ **ident** | ARRAY-TYPE | RANGE | ENUMERATED | RECORD-TYPE
10. ARRAY-TYPE $\rightarrow$ **arraysy** + **lbrack** + (RANGE | **ident**) + **rbrack** + **ofsy** + TYPE
11. RANGE $\rightarrow$ CONSTANT + **period** + **period** + CONSTANT
12. ENUMERATED $\rightarrow$ **lparent** + **ident** + (**comma** + **ident**)* + **rparent**
13. RECORD-TYPE $\rightarrow$ **recordsy** + FIELD-LIST + **endsy**
14. FIELD-LIST $\rightarrow$ FIELD-PART + (**semicolon** + FIELD-PART)*
15. FIELD-PART $\rightarrow$ IDENTIFIER-LIST + **colon** + TYPE
16. SUBPROGRAM-DECLARATION $\rightarrow$ PROCEDURE-DECLARATION | FUNCTION-DECLARATION
17. PROCEDURE-DECLARATION $\rightarrow$ **proceduresy** + **ident** + (FORMAL-PARAMETER-LIST)? + **semicolon** + BLOCK + **semicolon**
18. FUNCTION-DECLARATION $\rightarrow$ **functionsy** + **ident** + (FORMAL-PARAMETER-LIST)? + **colon** + **ident** + **semicolon** + BLOCK + **semicolon**
19. BLOCK $\rightarrow$ DECLARATION-PART + COMPOUND-STATEMENT
20. FORMAL-PARAMETER-LIST $\rightarrow$ **lparent** + PARAMETER-GROUP + (**semicolon** + PARAMETER-GROUP)* + **rparent**
21. PARAMETER-GROUP $\rightarrow$ IDENTIFIER-LIST + **colon** + (**ident** | ARRAY-TYPE)
22. COMPOUND-STATEMENT $\rightarrow$ **beginsy** + STATEMENT-LIST + **endsy**
23. STATEMENT-LIST $\rightarrow$ STATEMENT (**semicolon** + STATEMENT)*
24. STATEMENT $\rightarrow$ (ASSIGNMENT-STATEMENT | IF-STATEMENT | CASE-STATEMENT | WHILE-STATEMENT | REPEAT-STATEMENT | FOR-STATEMENT | PROCEDURE/FUNCTION-CALL)?
25. VARIABLE $\rightarrow$ **ident** + (COMPONENT-VARIABLE)*
26. COMPONENT-VARIABLE $\rightarrow$ (**lbrack** + INDEX-LIST + **rbrack**) | (**period** + **ident**)
27. INDEX-LIST $\rightarrow$ (**intcon** | **charcon** | **ident**) + (**comma** + INDEX-LIST)*
28. ASSIGNMENT-STATEMENT $\rightarrow$ VARIABLE + **becomes** + EXPRESSION
29. IF-STATEMENT $\rightarrow$ **ifsy** + EXPRESSION + **thensy** + STATEMENT + (**elsy** + STATEMENT)?
30. CASE-STATEMENT $\rightarrow$ **casesy** + EXPRESSION + **ofsy** + CASE-BLOCK + **endsy**
31. CASE-BLOCK $\rightarrow$ CONSTANT + (**comma** + CONSTANT)* + **colon** + STATEMENT + (**semicolon** + CASE-BLOCK)?*
32. WHILE-STATEMENT $\rightarrow$ **whilesy** + EXPRESSION + **dosy** + STATEMENT
33. REPEAT-STATEMENT $\rightarrow$ **repeatsy** + STATEMENT-LIST + **untilsy** + EXPRESSION
34. FOR-STATEMENT $\rightarrow$ **forsy** + **ident** + **becomes** + EXPRESSION + (**tosy** | **downtosy**) + EXPRESSION + **dosy** + STATEMENT
35. PROCEDURE/FUNCTION-CALL $\rightarrow$ **ident** + (**lparent** + PARAMETER-LIST? + **rparent**)?
36. PARAMETER-LIST $\rightarrow$ EXPRESSION (**comma** + EXPRESSION)*
37. EXPRESSION $\rightarrow$ SIMPLE-EXPRESSION (RELATIONAL-OPERATOR + SIMPLE-EXPRESSION)?
38. SIMPLE-EXPRESSION $\rightarrow$ (**plus** | **minus**)? TERM (ADDITIVE-OPERATOR + TERM)*
39. TERM $\rightarrow$ FACTOR (MULTIPLICATIVE-OPERATOR + FACTOR)*
40. FACTOR $\rightarrow$ **ident** | **realcon** | **intcon** | **charcon** | **string** | (**lparent** + EXPRESSION + **rparent**) | **notsy** + FACTOR | PROCEDURE/FUNCTION-CALL | VARIABLE
41. RELATIONAL-OPERATOR $\rightarrow$ **eql** | **neq** | **gtr** | **geq** | **lss** | **leq**
42. ADDITIVE-OPERATOR $\rightarrow$ **plus** | **minus** | **orsy**

1.4. Algoritma Recursive Descent Parsing

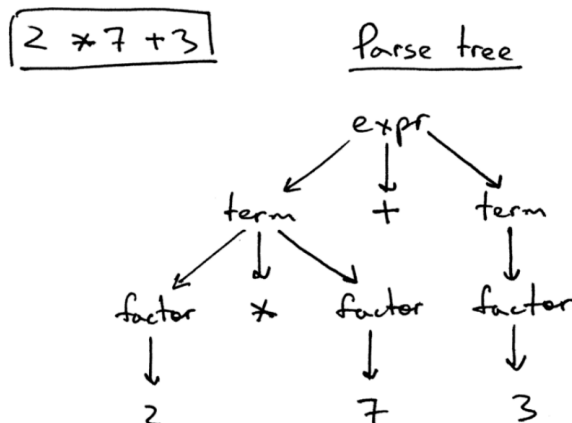
Algoritma *Recursive Descent Parsing* merupakan salah satu jenis *Top-down parser*, yaitu proses *parsing* yang dimulai dari *root* (akar) dan tumbuh ke bagian daun. Pada setiap langkahnya, *parser* akan memilih *node* untuk sebuah simbol non-terminal, lalu mengekspansinya dengan *subtree* yang merepresentasikan aturan produksinya. Proses ini menggunakan fungsi rekursif dan terus berlanjut hingga semua cabang mencapai simbol terminal.

Algoritma Recursive Descent ini memanfaatkan CFG dengan pendekatan top-down parsing sehingga mengurangi backtracking bahkan tidak melakukan backtrack sama sekali. Algoritma ini akan berhenti jika token sudah habis dibaca atau jika menemukan ketidakcocokan (sintaks error).

Disini kami menggunakan imlementasi dari *Hand-coded recursive-descent parsers* yaitu *Parser* yang ditulis secara manual oleh manusia menggunakan fungsi rekursif.

1.5. Parse Tree

Parse Tree adalah representasi lengkap dari struktur sintaks program sesuai dengan grammar bahasa yang digunakan. Setiap node di dalam parse tree merepresentasikan **produksi grammar**, sehingga pohon ini memuat semua simbol terminal dan non-terminal.



Gambar Parse Tree

(Sumber: <https://ruslanspivak.com/lbasi-part7>)

1.6. Keterkaitan Lexer dengan Parser

Lexer yang sudah diimplementasikan pada milestone sebelumnya memecah kode program menjadi urutan token-token yang kemudian akan diproses oleh parser untuk memastikan urutan token membentuk struktur sintaks yang valid, mendeteksi dan melaporkan kesalahan sintaks (syntax error) serta mempersiapkan representasi program untuk tahap berikutnya (Semantic Analysis).

2. Perancangan dan Implementasi

2.1. Teknologi yang digunakan

Program ini dibangun menggunakan bahasa C++ dengan memanfaatkan Standard Template Library (STL) seperti `std::vector`, `std::string`, dan `std::shared_ptr`. Program juga menggunakan `makefile` untuk mengotomatisasi proses kompilasi program. Alasan penggunaan bahasa C++ beserta STL untuk memaksimalkan performa dan menjamin keamanan memori, serta konfigurasi dengan `makefile` untuk memastikan siklus kompilasi yang otomatis, konsisten, dan efisien.

2.2. Arsitektur Program

Program dapat dibagi menjadi beberapa komponen utama yang masing-masing memiliki tanggung jawab spesifik yaitu:

1. Input

Input bertugas menerima masukan berupa source file yang akan dianalisis oleh program. Komponen akan menerima input source file dari pengguna lalu akan memvalidasi akses file. Setelah file berhasil diterima, komponen melakukan pembacaan isi file secara keseluruhan

2. Lexical Analyzer

Lexical analyzer bertugas mengubah rangkaian karakter dari source file menjadi sekumpulan token yang memiliki makna sintaksis. Proses yang dilakukan lexer meliputi membaca karakter dari buffer input, mengelompokkan karakter menjadi lexeme, dan mengklasifikasikan lexeme berdasarkan aturan token. Jika ditemukan karakter atau pola yang tidak sesuai dengan aturan lexical, lexer akan menghasilkan lexical error.

3. Syntax Analyzer

Syntax analyzer bertugas menerima token hasil keluaran lexical analyzer dan memvalidasinya berdasarkan grammar yang telah didefinisikan. Syntax analyzer bekerja dengan cara memeriksa apakah urutan token membentuk struktur yang valid sesuai aturan produksi grammar.

4. Output

Output bertugas menampilkan hasil akhir dari proses parsing kepada pengguna. Output yang dihasilkan berupa informasi detail mengenai token atau hasil parsing, representasi parse tree, dan pesan error beserta lokasi kesalahan

Pilihan arsitektur yang membagi program menjadi beberapa komponen utama digunakan untuk menjamin high cohesion dan loose coupling dalam program sehingga memudahkan proses isolasi kesalahan, pemeliharaan, serta skalabilitas pengembangan di masa depan. Pemisahan tanggung jawab yang tegas ini sejalan dengan prinsip desain *single responsibility principle*, yaitu Lexer berfokus eksklusif pada pembentukan dan validasi token dari teks mentah secara linear, sedangkan Parser dapat beroperasi secara lebih abstrak untuk mengevaluasi parse tree berdasarkan grammar tanpa perlu dibebani oleh kerumitan penanganan karakter, memungkinkan setiap tahap kompilasi diuji secara mandiri.

2.3. Representasi Parse Tree

Parse Tree yang dihasilkan oleh parser di program ini direpresentasikan menggunakan struktur data `TreeNode` yang berisi kumpulan data `NodeType`. Struktur data `TreeNode` yang digunakan sebagai berikut

- `NodeType`: sebuah enum yang menandai identitas node, meliputi apakah ini sebuah `Variable`, `IfStatement`, atau `Expression`.
- `value`: menyimpan teks asli dari kode sumber, misalnya nama variabel "x" atau angka "10".
- `children`: daftar yang berisi pointer ke node anak di bawahnya.

Representasi parse tree mengikuti logika Top-Down, yaitu:

- **Node Akar (Root)**: selalu berupa `<program>` sebagai tanda awal dari program.
- **Node Cabang**: merepresentasikan aturan tata bahasa. Contoh: Node `IfStatement` akan selalu memiliki anak berupa `Expression` (kondisi), `Statement` (jalur then), dan opsional `Statement` (jalur else).
- **Node Daun (Leaf)**: merupakan token akhir yang tidak punya anak lagi, seperti `IDENT`, `INTCON`, atau simbol operator.

Berikut implementasi dari representasi parse tree

```
class TreeNode {
public:
    NodeType type;
    std::string value;
    std::vector<std::shared_ptr<TreeNode>> children;

    TreeNode(NodeType, const std::string& = "");
    void addChild(const std::shared_ptr<TreeNode>& child);

    std::string typeString() const {
        return nodeTypeToString(type);
    }
};

// Konstruktor
TreeNode::TreeNode(const NodeType type_, const std::string& value_) :
type(type_), value(value_)
{
}

void TreeNode::addChild(const std::shared_ptr<TreeNode>& child) {
    if (child == nullptr) return;
    children.push_back(child);
}
```

```
enum class NodeType {
    Program,
    ProgramHeader,
    DeclarationPart,          // done <DeclarationPart> ->
    <ConstDeclaration> <TypeDeclaration> <VarDeclaration>
    <SubprogramDeclaration>
    ConstDeclaration,        // done <ConstDeclaration> -> const
    <Constant> {; <Constant>}
    Constant,                // nope <Constant> -> ident = (number |
char | string)
    TypeDeclaration,         // nope <TypeDeclaration> -> type ident =
    <Type>
    VarDeclaration,          // nope <VarDeclaration> -> var
    <IdentifierList> : <Type> {; <IdentifierList> : <Type>}
    IdentifierList,          // nope <IdentifierList> -> ident {, ident}
    Type,                    // nope <Type> -> <ArrayType> | <Range> |
    <Enumerated> | <RecordType> | ident
    // ...
};
```

```
inline const char* nodeTypeToString(NodeType type) {
    switch (type) {
        case NodeType::Program: return "<program>";
        case NodeType::ProgramHeader: return "<program-header>";
        case NodeType::DeclarationPart: return "<declaration-part>";
        case NodeType::ConstDeclaration: return "<const-declaration>";
        case NodeType::Constant: return "<constant>";
    }
    // ...
}
```

Ketika ditampilkan atau di-print ke terminal, parse tree dari program akan berbentuk seperti ini

```

<program> // root
├── <program-header> // child pertama
│   ├── programsy(program) // child kedua sekaligus leaf
│   ├── ident(Test1)
│   └── semicolon(;)
├── <declaration-part> // child pertama
│   └── <const-declaration>
└── .....
    ├── .....
    └── .....
        ├── endsy(end)
        └── period(.)

```

2.4. Kelas Parser dan Mekanisme Recursive Descent

Kelas Parser merupakan komponen utama dalam program yang bertugas utama untuk melakukan analisis sintaksis. Parser menerima deretan token dari Lexer dan memvalidasinya berdasarkan aturan tata bahasa yang telah ditentukan untuk bahasa Arion.

Mekanisme yang digunakan dalam parser ini adalah Recursive Descent, yaitu metode top-down parsing yang membangun parse tree dari root menuju leaf. Mekanisme ini bekerja dengan cara:

- Pemetaan Grammar ke Fungsi: Setiap aturan sintaksis (EBNF) direpresentasikan oleh satu fungsi anggota di dalam kelas Parser (contoh: `parseStatement()`, `parseExpression()`).
- Rekursi: Fungsi-fungsi tersebut saling memanggil secara rekursif untuk menangani struktur kode yang bersarang (nested), seperti ekspresi matematika di dalam tanda kurung atau pernyataan `if` di dalam `while`.

Kelas Parser mengelola posisi pembacaan token melalui variabel penunjuk (`pos`) dan menyimpan daftar token dalam sebuah vektor. Beberapa metode utama yang mendukung jalannya mekanisme ini adalah:

- `match()` : mengecek apakah token saat ini sesuai dengan tipe yang diharapkan tanpa memajukan posisi jika gagal.
- `expect()` : menagih keberadaan token tertentu. Jika tidak ditemukan, fungsi ini akan menambahkan syntax error yang ditemukan ke atribut `SyntaxError`
- `terminal()` : mencocokkan token dan langsung membungkusnya menjadi `TreeNode` sebagai leaf pada parse tree.
- `need()` : pembantu untuk mencoba memanggil fungsi parsing lain dan secara otomatis menambahkan hasilnya sebagai anak (`child`) dari node saat ini

Parser juga menerapkan konsep backtracking. Saat Parser mencoba mencocokkan sebuah aturan namun gagal di tengah jalan, Parser akan memanggil fungsi `save()` untuk mencatat posisi token saat itu dan `restore()` atau `fail()` untuk mengembalikan posisi kursor ke titik sebelum percobaan dilakukan. Hal ini memungkinkan Parser mencoba alternatif aturan lain tanpa merusak urutan pembacaan token.

Berikut implementasi kelas Parser

```
#include "Parser.hpp"
#include "iostream"

Parser::Parser(const std::vector<Token> &tokens)
    : tokens(tokens), pos(0) {}

bool Parser::match(TokenType type) const
{
    return pos < tokens.size() && tokens[pos].type == type;
}

size_t Parser::save() const
```

```

{
    return pos;
}

void Parser::restore(size_t saved)
{
    pos = saved;
}

std::shared_ptr<TreeNode> Parser::fail(size_t saved)
{
    restore(saved);
    return nullptr;
}

std::shared_ptr<TreeNode> Parser::terminal(TokenType tokenType, NodeType
nodeType)
{
    if (!match(tokenType)) {
        return nullptr;
    }

    auto value = tokens[pos].value;
    pos++;

    return std::make_shared<TreeNode>(nodeType, value);
}

std::shared_ptr<TreeNode> Parser::expect(TokenType tokenType, NodeType
nodeType, std::string expectedStr)
{
    if (match(tokenType)) {
        return terminal(tokenType, nodeType);
    }

    const Token &currentToken = peek();
    std::string detailMsg = "Diharapkan " + expectedStr + " tetapi menemukan
'" + currentToken.value + "'";
    SyntaxError err(detailMsg, currentToken.line, currentToken.column);
    syntaxErrors.push_back(err.what());
    std::shared_ptr<TreeNode> errorNode =
std::make_shared<TreeNode>(nodeType, "<Missing " + expectedStr + ">", true);
    return errorNode;
}

/**
 * Untuk melakukan Panic Mode Recovery
 *
 * So caranya jika ditemukan token yang salah maka
 * akan mencari token samapai menemukan token yang aman untuk melakukan
 * parsing berikutnya
 * Pijakan yang aman

```

```

* 1. Berakhir dari ;
* 2. Awalan dari statement baru if, while, var, begin
*/
void Parser::synchronize()
{
    advance();
    while (pos < tokens.size()) {
        // Jika token sebelumnya ; sudah melewati error
        if (peek(-1).type == TokenType::SEMICOLON) {
            return;
        }
        // jika token adalah awal dari statement atau blok baru,
        // berhenti membuang token
        switch (peek().type) {
            case TokenType::VARSY:
            case TokenType::CONSTSY:
            case TokenType::TYPESY:
            case TokenType::PROCEDURESY:
            case TokenType::FUNCTIONSY:
            case TokenType::BEGINSY:
            case TokenType::IFSY:
            case TokenType::WHILESY:
            case TokenType::REPEATSY:
            case TokenType::FORSY:
            case TokenType::CASESY:
                return;
            default:
                break;
        }
        advance();
    }
}

// this function checks whether the child exist or not
bool Parser::need(const std::shared_ptr<TreeNode> &parent, const
std::shared_ptr<TreeNode> &child)
{
    if (child == nullptr) {
        return false;
    }

    parent->addChild(child);
    return true;
}

// 1 PROGRAM → PROGRAM-HEADER + DECLARATION-PART + COMPOUND-STATEMENT +
period
std::shared_ptr<TreeNode> Parser::parseProgram()
{
    auto node = std::make_shared<TreeNode>(NodeType::Program);

```

```

    auto header = parseProgramHeader();
    if (header) node->addChild(header);

    auto declPart = parseDeclarationPart();
    if (declPart) node->addChild(declPart);

    auto compStmt = parseCompoundStatement();
    if (compStmt) node->addChild(compStmt);

    auto period = expect(TokenType::PERIOD, NodeType::Period, ".");
    need(node, period);

    return node;
}

// 2 PROGRAM-HEADER → programsy + ident + semicolon
std::shared_ptr<TreeNode> Parser::parseProgramHeader()
{
    auto node = std::make_shared<TreeNode>(NodeType::ProgramHeader);

    need(node, expect(TokenType::PROGRAMSY, NodeType::ProgramSy,
        "'program'"));
    need(node, expect(TokenType::IDENT, NodeType::Ident, "identifier
program"));
    auto semi = expect(TokenType::SEMICOLON, NodeType::Semicolon, ";");
    need(node, semi);

    if (semi->isError) {
        synchronize();
    }

    return node;
}

// 3 DECLARATION-PART → (CONST-DECLARATION)* + (TYPE-DECLARATION)* +
(VAR-DECLARATION)* + (SUBPROGRAM-DECLARATION)*
std::shared_ptr<TreeNode> Parser::parseDeclarationPart()
{
    auto node = std::make_shared<TreeNode>(NodeType::DeclarationPart);

    // a* + b* + c* + d*
    while (need(node, parseConstDeclaration()));
    while (need(node, parseTypeDeclaration()));
    while (need(node, parseVarDeclaration()));
    while (need(node, parseSubprogramDeclaration()));
    return node;
}

// 4 CONST-DECLARATION → constsy + (ident + eql + CONSTANT + semicolon)
std::shared_ptr<TreeNode> Parser::parseConstDeclaration()
{

```

```

        if (peek().type != TokenType::CONSTSY) return nullptr;
        auto node = std::make_shared<TreeNode>(NodeType::ConstDeclaration);
        need(node, terminal(TokenType::CONSTSY, NodeType::ConstSy));
        do {
            auto idNode = expect(TokenType::IDENT, NodeType::Ident, "identifier
konstanta");
            need(node, idNode);
            if (idNode->isError) synchronize();

            auto eqlNode = expect(TokenType::EQL, NodeType::Eql, "'=='");
            need(node, eqlNode);

            auto constVal = parseConstant();
            if (constVal) {
                need(node, constVal);
            }
            else {
                SyntaxError err("Diharapkan nilai konstanta", peek().line,
peek().column);
                syntaxErrors.push_back(err.what());

                auto errNode = std::make_shared<TreeNode>(NodeType::Constant,
"<Missing Constant>", true);
                node->addChild(errNode);
                synchronize();
            }

            auto semi = expect(TokenType::SEMICOLON, NodeType::Semicolon, "';");
            need(node, semi);

            // PENTING: Jika titik koma error, kita sync untuk mencoba memulihkan
            if (semi->isError) synchronize();

        } while (peek().type == TokenType::IDENT);

        return node;
    }

// 5 CONSTANT → charcon | string | [(plus | minus )? + (ident | intcon |
realcon)]
std::shared_ptr<TreeNode> Parser::parseConstant()
{
    auto node = std::make_shared<TreeNode>(NodeType::Constant);

    // Coba opsi yang valid satu per satu
    if (auto c = terminal(TokenType::CHARCON, NodeType::CharCon)) {
        node->addChild(c);
        return node;
    }
    if (auto s = terminal(TokenType::STRINGCON, NodeType::String)) {
        node->addChild(s);
    }
}

```



```

        return node;
    }

    // Jika ada plus/minus (opsional)
    if (auto p = terminal(TokenType::PLUS, NodeType::Plus))
node->addChild(p);
        else if (auto m = terminal(TokenType::MINUS, NodeType::Minus))
node->addChild(m);

    // Jika sudah ada plus/minus, atau kosongan, cari nilainya
    if (auto id = terminal(TokenType::IDENT, NodeType::Ident)) {
        node->addChild(id);
        return node;
    }
    if (auto ic = terminal(TokenType::INTCON, NodeType::IntCon)) {
        node->addChild(ic);
        return node;
    }
    if (auto rc = terminal(TokenType::REALCON, NodeType::RealCon)) {
        node->addChild(rc);
        return node;
    }

    // Jika kita sudah mengonsumsi plus/minus TAPI tidak menemukan
    angka/ident setelahnya
    if (!node->children.empty()) {
        SyntaxError err("Diharapkan angka atau identifier setelah tanda +/-",
peek().line, peek().column);
        syntaxErrors.push_back(err.what());
        node->addChild(std::make_shared<TreeNode>(NodeType::Error, "<Missing
Number>", true));
        return node;
    }
    // Jika sama sekali bukan konstanta, kembalikan nullptr
    return nullptr;
}

// 6 TYPE-DECLARATION → typesy + (ident + eql + TYPE + semicolon)+
TYPE-DECLARATION
std::shared_ptr<TreeNode> Parser::parseTypeDeclaration()
{
    if (peek().type != TokenType::TYPESY) return nullptr;

    auto node = std::make_shared<TreeNode>(NodeType::TypeDeclaration);
    need(node, terminal(TokenType::TYPESY, NodeType::TypeSy));

    do {
        auto idNode = expect(TokenType::IDENT, NodeType::Ident, "identifier
tipe");
        need(node, idNode);
        if (idNode->isError) synchronize();
    }

```

```

        auto eqlNode = expect(TokenType::EQL, NodeType::Eql, "'=='");
        need(node, eqlNode);
        auto typeNode = parseType();
        if (typeNode) {
            need(node, typeNode);
        }
        else {
            SyntaxError err("Diharapkan definisi tipe", peek().line,
peek().column);
            syntaxErrors.push_back(err.what());
            node->addChild(std::make_shared<TreeNode>(NodeType::Type,
"<Missing Type>", true));
            synchronize();
        }
        auto semi = expect(TokenType::SEMICOLON, NodeType::Semicolon, "';");
        need(node, semi);
        if (semi->isError) synchronize();
    } while (peek().type == TokenType::IDENT);

    return node;
}

// 7 VAR-DECLARATION → varsy + (IDENTIFIER-LIST + colon + TYPE + semicolon)
+ VAR-DECLARATION
std::shared_ptr<TreeNode> Parser::parseVarDeclaration()
{
    if (peek().type != TokenType::VARSY) return nullptr;

    auto node = std::make_shared<TreeNode>(NodeType::VarDeclaration);
    need(node, terminal(TokenType::VARSY, NodeType::VarSy));

    do {
        auto idList = parseIdentifierList();
        if (idList) need(node, idList);

        auto colonNode = expect(TokenType::COLON, NodeType::Colon, "':'");
        need(node, colonNode);

        auto typeNode = parseType();
        if (typeNode) {
            need(node, typeNode);
        }
        else {
            SyntaxError err("Diharapkan tipe data variabel", peek().line,
peek().column);
            syntaxErrors.push_back(err.what());
            node->addChild(std::make_shared<TreeNode>(NodeType::Type,
"<Missing Type>", true));
            synchronize();
        }
    }

```

```

        auto semi = expect(TokenType::SEMICOLON, NodeType::Semicolon, "';");
        need(node, semi);
        if (semi->isError) synchronize();

    } while (peek().type == TokenType::IDENT);

    return node;
}

// 8 IDENTIFIER-LIST → ident (comma + ident)*
std::shared_ptr<TreeNode> Parser::parseIdentifierList()
{
    auto node = std::make_shared<TreeNode>(NodeType::IdentifierList);

    auto firstId = expect(TokenType::IDENT, NodeType::Ident, "identifier");
    need(node, firstId);

    if (firstId->isError) return node;

    while (peek().type == TokenType::COMMA) {
        need(node, expect(TokenType::COMMA, NodeType::Comma, ","));
        need(node, expect(TokenType::IDENT, NodeType::Ident, "identifier"));
    }

    return node;
}

// 9 TYPE → ident | ARRAY-TYPE | RANGE | ENUMERATED | RECORD-TYPE
std::shared_ptr<TreeNode> Parser::parseType()
{
    auto node = std::make_shared<TreeNode>(NodeType::Type);

    if (peek().type == TokenType::ARRAYSY) {
        need(node, parseArrayType());
        return node;
    }

    auto rangeNode = parseRange();
    if (rangeNode) {
        need(node, rangeNode);
        return node;
    }

    auto enumNode = parseEnumerated();
    if (enumNode) {
        need(node, enumNode);
        return node;
    }

    auto recordNode = parseRecordType();
    if (recordNode) {

```

```

        need(node, recordNode);
        return node;
    }

    // Jika bukan array, range, enum, atau record, maka HARUS identifier
    // (misal tipe data primitif kustom atau bawaan spt 'integer', 'boolean')
    if (peek().type == TokenType::IDENT) {
        need(node, terminal(TokenType::IDENT, NodeType::Ident));
        return node;
    }

    // Jika sampai di sini, berarti bukan tipe data valid sama sekali
    return nullptr;
}

// 10 ARRAY-TYPE → arraysy + lbrack + (RANGE | ident) + rbrack + ofsy + TYPE
std::shared_ptr<TreeNode> Parser::parseArrayType()
{
    if (peek().type != TokenType::ARRAYSY) return nullptr;

    auto node = std::make_shared<TreeNode>(NodeType::ArrayType);

    need(node, terminal(TokenType::ARRAYSY, NodeType::ArraySy));

    auto lBrack = expect(TokenType::LBRACK, NodeType::LBrack, "'['");
    need(node, lBrack);
    if (lBrack->isError) synchronize();

    auto rangeNode = parseRange();
    if (rangeNode) {
        need(node, rangeNode);
    }
    else {
        auto idNode = expect(TokenType::IDENT, NodeType::Ident, "range atau identifier");
        need(node, idNode);
    }

    auto rBrack = expect(TokenType::RBRACK, NodeType::RBrack, "']'");
    need(node, rBrack);
    if (rBrack->isError) synchronize();

    auto ofNode = expect(TokenType::OFSY, NodeType::OfSy, "'of'");
    need(node, ofNode);

    auto typeNode = parseType();
    if (typeNode) {
        need(node, typeNode);
    }
    else {
        SyntaxError err("Diharapkan tipe data untuk elemen array",

```

```

peek().line, peek().column);
    syntaxErrors.push_back(err.what());
    node->addChild(std::make_shared<TreeNode>(NodeType::Type, "<Missing
Array Type>", true));
}

    return node;
}

// 11 RANGE → CONSTANT + period + period + CONSTANT
std::shared_ptr<TreeNode> Parser::parseRange()
{
    size_t saved = save();
    auto const1 = parseConstant();
    if (!const1 || peek().type != TokenType::PERIOD) {
        restore(saved);
        return nullptr;
    }
    auto node = std::make_shared<TreeNode>(NodeType::Range);
    need(node, const1);

    auto periodNode1 = expect(TokenType::PERIOD, NodeType::Period, ".");
    need(node, periodNode1);
    if (periodNode1->isError) synchronize();

    auto periodNode2 = expect(TokenType::PERIOD, NodeType::Period, ".");
    need(node, periodNode2);
    if (periodNode2->isError) synchronize();

    auto const2 = parseConstant();
    if (const2) {
        need(node, const2);
    }
    else {
        SyntaxError err("Diharapkan Konstanta (batas) di baris
", peek().line, peek().column);
        syntaxErrors.push_back(err.what());
        node->addChild(std::make_shared<TreeNode>(NodeType::Error, "<Missing
Constant>", true));
        synchronize();
    }
    return node;
}

// 12 ENUMERATED → lparent + ident + (comma + ident)* + rparent
std::shared_ptr<TreeNode> Parser::parseEnumerated()
{
    if (peek().type != TokenType::LPARENT) {
        return nullptr;
    }

```

```

    auto node = std::make_shared<TreeNode>(NodeType::Enumerated);
    auto lpar = expect(TokenType::LPARENT, NodeType::LParent, "'('");
    need(node, lpar);

    auto ident = expect(TokenType::IDENT, NodeType::Ident, "enumerated
identifier" );
    need(node, ident);
    if (ident->isError) synchronize();

    while (peek().type == TokenType::COMMA) {
        need(node, terminal(TokenType::COMMA, NodeType::Comma));

        auto nextIden = expect(TokenType::IDENT, NodeType::Ident, "enumerated
identifier setelah koma");
        need(node, nextIden);
        if (nextIden->isError) synchronize();
    }

    auto rpar = expect(TokenType::RPARENT, NodeType::RParent, "')'");
    need(node, rpar);
    if (rpar->isError) synchronize();
    return node;
}

// 13 RECORD-TYPE → recordsy + FIELD-LIST + endsy
std::shared_ptr<TreeNode> Parser::parseRecordType()
{
    if (peek().type != TokenType::RECORDSY) {
        return nullptr;
    }

    auto node = std::make_shared<TreeNode>(NodeType::RecordType);

    auto recordsy = expect(TokenType::RECORDSY, NodeType::RecordSy, "record
type");
    need(node, recordsy);

    if (recordsy->isError) synchronize();
    auto fieldList= parseFieldList();
    if (fieldList){
        need(node, fieldList);
    }else{
        SyntaxError err("Diharapkan daftar field", peek().line,
peek().column);
        syntaxErrors.push_back(err.what());
        node->addChild(std::make_shared<TreeNode>(NodeType::Error, "<Missing
Field-List>", true));
        synchronize();
    }

    auto endsy = expect(TokenType::ENDSY, NodeType::EndSy, "end");

```

```

        need(node, endsy);
        if (endsy->isError){
            synchronize();
        }
        return node;
    }
}

// 14 FIELD-LIST → FIELD-PART + (semicolon + FIELD-PART)*
std::shared_ptr<TreeNode> Parser::parseFieldList()
{
    auto node = std::make_shared<TreeNode>(NodeType::FieldList);

    auto firstField = parseFieldPart();
    if (firstField)
    {
        need(node, firstField);
    }else{
        return nullptr;
    }

    while (peek().type == (TokenType::SEMICOLON)) {
        need(node, terminal(TokenType::SEMICOLON, NodeType::Semicolon));

        auto nextField = parseFieldPart();
        if (nextField){
            need(node, nextField);
        }else{
            SyntaxError err("Diharapkan field-part setelah ';'", peek().line,
peek().column);
            syntaxErrors.push_back(err.what());
            node->addChild(std::make_shared<TreeNode>(NodeType::Error,
"<Missing Field-Part>", true));
            synchronize();
        }
    }

    return node;
}

// 15 FIELD-PART → IDENTIFIER-LIST+ colon + TYPE
std::shared_ptr<TreeNode> Parser::parseFieldPart()
{
    auto node = std::make_shared<TreeNode>(NodeType::FieldPart);
    auto idenList = parseIdentifierList();
    if (idenList){
        need(node, idenList);
    }else{
        SyntaxError err("Diharapkan daftar Identifier", peek().line,
peek().column);
        syntaxErrors.push_back(err.what());
        node->addChild(std::make_shared<TreeNode>(NodeType::Error, "<Missing
IdentifierList>", true));
    }
}

```

```

        synchronize();
    }

    auto colon = expect(TokenType::COLON, NodeType::Colon, ":");
    need(node, colon);
    if (colon->isError){
        synchronize();
    }

    auto type = parseType();
    if (type)
    {
        need(node, type);
    }else{
        SyntaxError err("Diharapkan Type Data", peek().line, peek().column);
        syntaxErrors.push_back(err.what());
        node->addChild(std::make_shared<TreeNode>(NodeType::Error, "<Missing
Type>", true));
        synchronize();
    }
    return node;
}

// 16 SUBPROGRAM-DECLARATION -> PROCEDURE-DECLARATION | FUNCTION-DECLARATION

std::shared_ptr<TreeNode> Parser::parseSubprogramDeclaration()
{
    if (peek().type == TokenType::PROCEDURESY) {
        return parseProcedureDeclaration();
    } else if (peek().type == TokenType::FUNCTIONSY) {
        return parseFunctionDeclaration();
    }

    return nullptr;
}

// 17 PROCEDURE-DECLARATION -> proceduresy + ident + (FORMAL-PARAMETER-LIST)?
+ semicolon + BLOCK + semicolon
std::shared_ptr<TreeNode> Parser::parseProcedureDeclaration()
{
    auto node = std::make_shared<TreeNode>(NodeType::ProcedureDeclaration);
    need(node, terminal(TokenType::PROCEDURESY, NodeType::ProcedureSy));

    auto idNode = expect(TokenType::IDENT, NodeType::Ident, "nama
procedure");
    need(node, idNode);
    if (idNode->isError) synchronize();

    auto params = parseFormalParameterList();
    if (params) need(node, params);
}

```



```

    auto semil = expect(TokenType::SEMICOLON, NodeType::Semicolon, "';");
    need(node, semil);
    if (semil->isError) synchronize();

    auto blockNode = parseBlock();
    if (blockNode) {
        need(node, blockNode);
    } else {
        SyntaxError err("Diharapkan blok kode (begin...end) pada procedure",
peek().line, peek().column);
        syntaxErrors.push_back(err.what());
        node->addChild(std::make_shared<TreeNode>(NodeType::Error, "<Missing
Block>", true));
        synchronize();
    }

    auto semi2 = expect(TokenType::SEMICOLON, NodeType::Semicolon, "';' di
akhir procedure");
    need(node, semi2);
    if (semi2->isError) synchronize();

    return node;
}

// 18 FUNCTION-DECLARATION -> functionsy + ident + (FORMAL-PARAMETER-LIST)? +
colon + ident + semicolon+ BLOCK + semicolon
std::shared_ptr<TreeNode> Parser::parseFunctionDeclaration()
{
    auto node = std::make_shared<TreeNode>(NodeType::FunctionDeclaration);

    need(node, terminal(TokenType::FUNCTIONSY, NodeType::FunctionSy));

    auto idNode = expect(TokenType::IDENT, NodeType::Ident, "nama function");
    need(node, idNode);
    if (idNode->isError) synchronize();

    // Opsional
    auto params = parseFormalParameterList();
    if (params) need(node, params);

    auto colonNode = expect(TokenType::COLON, NodeType::Colon, "':'");
    need(node, colonNode);
    if (colonNode->isError) synchronize();

    auto retType = expect(TokenType::IDENT, NodeType::Ident, "tipe kembalian
function");
    need(node, retType);
    if (retType->isError) synchronize();

    auto semil = expect(TokenType::SEMICOLON, NodeType::Semicolon, "';");
    need(node, semil);

```

```

        if (semi1->isError) synchronize();

        auto blockNode = parseBlock();
        if (blockNode) need(node, blockNode);

        auto semi2 = expect(TokenType::SEMICOLON, NodeType::Semicolon, "';' di
akhir function");
        need(node, semi2);
        if (semi2->isError) synchronize();

        return node;
    }

// 19 BLOCK -> DECLARATION-PART + COMPOUND-STATEMENT
std::shared_ptr<TreeNode> Parser::parseBlock()
{
    auto node = std::make_shared<TreeNode>(NodeType::Block);

    auto declPart = parseDeclarationPart();
    if (declPart) need(node, declPart);

    auto compStmt = parseCompoundStatement();
    if (compStmt) {
        need(node, compStmt);
    } else {
        SyntaxError err("Diharapkan blok 'begin ... end' (Compound
Statement)", peek().line, peek().column);
        syntaxErrors.push_back(err.what());
        node->addChild(std::make_shared<TreeNode>(NodeType::Error, "<Missing
Compound-Statement>", true));
        if (peek().type != TokenType::SEMICOLON) {
            synchronize();
        }
    }

    return node;
}

// 20 FORMAL-PARAMETER-LIST -> lparent + PARAMETER-GROUP + (semicolon +
PARAMETER-GROUP)* + rparent
std::shared_ptr<TreeNode> Parser::parseFormalParameterList()
{
    if (peek().type != TokenType::LPARENT) {
        return nullptr;
    }

    auto node = std::make_shared<TreeNode>(NodeType::FormalParameterList);

    auto lpar = expect(TokenType::LPARENT, NodeType::LParent, "'('");
    need(node, lpar);

```

```

    auto paramGroup = parseParameterGroup();
    if (paramGroup) {
        need(node, paramGroup);
    } else {
        SyntaxError err("Diharapkan definisi parameter", peek().line,
peek().column);
        syntaxErrors.push_back(err.what());
        node->addChild(std::make_shared<TreeNode>(NodeType::Error, "<Missing
ParameterGroup>", true));
        synchronize();
    }

    while (peek().type == TokenType::SEMICOLON) {
        need(node, terminal(TokenType::SEMICOLON, NodeType::Semicolon)); //
Konsumsi ';'

        auto nextParamGroup = parseParameterGroup();
        if (nextParamGroup) {
            need(node, nextParamGroup);
        } else {
            SyntaxError err("Diharapkan parameter setelah ';'", peek().line,
peek().column);
            syntaxErrors.push_back(err.what());
            node->addChild(std::make_shared<TreeNode>(NodeType::Error,
"<Missing ParameterGroup>", true));
            synchronize();
        }
    }

    auto rpar = expect(TokenType::RPARENT, NodeType::RParent, "')'");
    need(node, rpar);
    if (rpar->isError) synchronize();

    return node;
}

// 21 PARAMETER-GROUP -> IDENTIFIER-LIST + colon + (ident | ARRAY-TYPE)
std::shared_ptr<TreeNode> Parser::parseParameterGroup()
{
    size_t saved = save();
    auto node = std::make_shared<TreeNode>(NodeType::ParameterGroup);

    if (!need(node, parseIdentifierList())) return fail(saved);
    if (!need(node, terminal(TokenType::COLON, NodeType::Colon))) return
fail(saved);
    if (!need(node, terminal(TokenType::IDENT, NodeType::Ident)) ||
need(node, parseArrayType())) return fail(saved);

    return node;
}

```

```

// 22 COMPOUND-STATEMENT -> beginsy + STATEMENT-LIST + endsy
std::shared_ptr<TreeNode> Parser::parseCompoundStatement()
{
    if (peek().type != TokenType::BEGINSY) {
        return nullptr;
    }
    auto node = std::make_shared<TreeNode>(NodeType::CompoundStatement);
    need(node, terminal(TokenType::BEGINSY, NodeType::BeginSy));
    auto stmtList = parseStatementList();
    if (stmtList) {
        need(node, stmtList);
    } else {
        SyntaxError err("Diharapkan statement di dalam blok 'begin'",
peek().line, peek().column);
        syntaxErrors.push_back(err.what());
        node->addChild(std::make_shared<TreeNode>(NodeType::Error, "<Missing
Statement-List>", true));
        synchronize();
    }
    auto endSy = expect(TokenType::ENDSY, NodeType::EndSy, "'end'");
    need(node, endSy);
    if (endSy->isError) {
        synchronize();
    }
    return node;
}

// 23 STATEMENT-LIST -> STATEMENT (semicolon + STATEMENT)*
std::shared_ptr<TreeNode> Parser::parseStatementList()
{
    auto node = std::make_shared<TreeNode>(NodeType::StatementList);
    auto firstStatement = parseStatement();
    if (firstStatement) {
        need(node, firstStatement);
    } else {
        return nullptr;
    }

    while (peek().type == TokenType::SEMICOLON) {
        need(node, terminal(TokenType::SEMICOLON, NodeType::Semicolon)); //
Aman memakan token di sini

        auto nextStatement = parseStatement();
        if (nextStatement) {
            need(node, nextStatement);
        } else {
            SyntaxError err("Diharapkan statement setelah ';'", peek().line,
peek().column);
            syntaxErrors.push_back(err.what());
            node->addChild(std::make_shared<TreeNode>(NodeType::Error,
"<Missing Statement>", true));

```

```

        synchronize();
    }
}
return node;
}

// 24 STATEMENT -> (ASSIGNMENT-STATEMENT | IF-STATEMENT | CASE-STATEMENT |
// WHILE-STATEMENT | REPEAT-STATEMENT | FOR-STATEMENT )? |
// PROCEDURE/FUNCTION-CALL
std::shared_ptr<TreeNode> Parser::parseStatement()
{
    auto node = std::make_shared<TreeNode>(NodeType::Statement);
    std::shared_ptr<TreeNode> child = nullptr;

    TokenType type = peek().type;

    if (type == TokenType::IFSY)      child = parseIfStatement();
    else if (type == TokenType::CASESY) child = parseCaseStatement();
    else if (type == TokenType::WHILESY) child = parseWhileStatement();
    else if (type == TokenType::REPEATSY) child = parseRepeatStatement();
    else if (type == TokenType::FORSY)  child = parseForStatement();
    else if (type == TokenType::IDENT) {
        size_t saved = save();
        auto varNode = parseVariable();

        if (peek().type == TokenType::BECOMES) {
            restore(saved);
            child = parseAssignmentStatement();
        } else {
            restore(saved);
            child = parseProcedureCall();
        }
    }

    if (child) {
        need(node, child);
        return node;
    }

    return node;
}

// 25 ASSIGNMENT-STATEMENT -> ident + VARIABLE + becomes + EXPRESSION
std::shared_ptr<TreeNode> Parser::parseAssignmentStatement()
{
    if (peek().type != TokenType::IDENT) {
        return nullptr;
    }

    auto node = std::make_shared<TreeNode>(NodeType::AssignmentStatement);

    auto var = parseVariable();

```

```

        if (var){
            need(node, var);
        }else{
            SyntaxError err("Diharapkan Variable untuk diassign", peek().line,
peek().column);
            syntaxErrors.push_back(err.what());
            node->addChild(std::make_shared<TreeNode>(NodeType::Error, "<Missing
Variable>", true));
            synchronize();
        }

        auto becomesy = expect(TokenType::BECOMES, NodeType::Becomes, ":=");
        need(node, becomesy);
        if (becomesy->isError){
            synchronize();
        }
        auto expression = parseExpression();
        if (expression){
            need(node, expression);
        }else{
            SyntaxError err("Diharapkan ada ekspresi untuk diassign",
peek().line, peek().column);
            syntaxErrors.push_back(err.what());
            node->addChild(std::make_shared<TreeNode>(NodeType::Error, "<Missing
Expression>", true));
            synchronize();
        }

        return node;
    }

// 26 IF-STATEMENT -> ifsy + EXPRESSION + thensy + STATEMENT + (elsy +
STATEMENT)?
std::shared_ptr<TreeNode> Parser::parseIfStatement()
{
    if (peek().type != TokenType::IFSY){
        return nullptr;
    }
    auto node = std::make_shared<TreeNode>(NodeType::IfStatement);
    auto ifsy = terminal(TokenType::IFSY, NodeType::IfSy);
    need(node, ifsy);

    auto expression = parseExpression();
    if (expression){
        need(node, expression);
    }else{
        SyntaxError err("Diharapkan ekspresi kondisi setelah 'if'",
peek().line, peek().column);
        syntaxErrors.push_back(err.what());
        node->addChild(std::make_shared<TreeNode>(NodeType::Error, "<Missing

```

```

Expression>", true));
    synchronize();
}
auto thenSy = expect(TokenType::THENSY, NodeType::ThenSy, "'then'");
need(node, thenSy);
if (thenSy->isError){
    synchronize();
}
auto statement = parseStatement();
if (statement){
    need(node, statement);
}else{
    SyntaxError err("Diharapkan statement setelah 'then'", peek().line,
peek().column);
    syntaxErrors.push_back(err.what());
    node->addChild(std::make_shared<TreeNode>(NodeType::Error, "<Missing
Statement>", true));
    synchronize();
}

if (match(TokenType::ELSESY)) {
    auto elseSy = terminal(TokenType::ELSESY, NodeType::ElseSy);
    need(node, elseSy);

    auto elseStatement = parseStatement();
    if (elseStatement) {
        need(node, elseStatement);
    } else {
        SyntaxError err("Diharapkan statement setelah 'else'",
peek().line, peek().column);
        syntaxErrors.push_back(err.what());
        node->addChild(std::make_shared<TreeNode>(NodeType::Error,
"<Missing Else-Statement>", true));
        synchronize();
    }
}

return node;
}

// 27 CASE-STATEMENT -> casesy + EXPRESSION + ofsy + CASE-BLOCK + endsy
std::shared_ptr<TreeNode> Parser::parseCaseStatement()
{
    if (peek().type != TokenType::CASESY){
        return nullptr;
    }
    auto node = std::make_shared<TreeNode>(NodeType::CaseStatement);

    auto casesy = terminal(TokenType::CASESY, NodeType::CaseSy);
    need(node, casesy);

```

```

    auto expression = parseExpression();
    if (expression){
        need(node, expression);
    }else{
        SyntaxError err("Diharapkan ada ekspresi setelah 'case'",
peek().line, peek().column);
        syntaxErrors.push_back(err.what());
        node->addChild(std::make_shared<TreeNode>(NodeType::Error, "<Missing
Expression>", true));
        synchronize();
    }

    auto ofsy = expect(TokenType::OFSY, NodeType::OfSy, "of");
    need(node, ofsy);
    if (ofsy->isError){
        synchronize();
    }

    auto caseBlock = parseCaseBlock();
    if (caseBlock){
        need(node, caseBlock);
    }else{
        SyntaxError err("Diharapkan ada Case Block setelah 'of'",
peek().line, peek().column);
        syntaxErrors.push_back(err.what());
        node->addChild(std::make_shared<TreeNode>(NodeType::Error, "<Missing
Case Block>", true));
        synchronize();
    }

    auto endsy = expect(TokenType::ENDSY, NodeType::EndSy, "end");
    need(node, endsy);
    if (endsy->isError){
        synchronize();
    }

    return node;
}

// 28 CASE-BLOCK -> CONSTANT + (comma + CONSTANT)* + colon + STATEMENT +
(semicolon + CASE-BLOCK?)*
std::shared_ptr<TreeNode> Parser::parseCaseBlock()
{
    auto firstConst = parseConstant();
    if (!firstConst) return nullptr;

    auto node = std::make_shared<TreeNode>(NodeType::CaseBlock);
    need(node, firstConst);

    while (peek().type == TokenType::COMMA) {
        need(node, terminal(TokenType::COMMA, NodeType::Comma));
    }

```



```

        auto nextConst = parseConstant();
        if (nextConst) {
            need(node, nextConst);
        } else {
            SyntaxError err("Diharapkan konstanta setelah ' '", peek().line,
peek().column);
            syntaxErrors.push_back(err.what());
            node->addChild(std::make_shared<TreeNode>(NodeType::Error,
"<Missing Constant>", true));
            synchronize();
        }
    }

    auto colon = expect(TokenType::COLON, NodeType::Colon, "':'");
    need(node, colon);
    if (colon->isError) synchronize();

    auto stmt = parseStatement();
    if (stmt) {
        need(node, stmt);
    } else {
        SyntaxError err("Diharapkan statement setelah ':'", peek().line,
peek().column);
        syntaxErrors.push_back(err.what());
        node->addChild(std::make_shared<TreeNode>(NodeType::Error, "<Missing
Statement>", true));
        synchronize();
    }

    while (peek().type == TokenType::SEMICOLON) {
        need(node, terminal(TokenType::SEMICOLON, NodeType::Semicolon));

        auto nextCase = parseCaseBlock();
        if (nextCase) {
            need(node, nextCase);
        }
    }

    return node;
}

// 29 WHILE-STATEMENT -> whilesy + EXPRESSION + dosy + STATEMENT
std::shared_ptr<TreeNode> Parser::parseWhileStatement()
{
    if (peek().type != TokenType::WHILESY) return nullptr;

    auto node = std::make_shared<TreeNode>(NodeType::WhileStatement);
    need(node, terminal(TokenType::WHILESY, NodeType::WhileSy));

    auto expr = parseExpression();

```

```

    if (expr) {
        need(node, expr);
    } else {
        SyntaxError err("Diharapkan ekspresi kondisi setelah 'while'",
peek().line, peek().column);
        syntaxErrors.push_back(err.what());
        node->addChild(std::make_shared<TreeNode>(NodeType::Error, "<Missing
Expression>", true));
        if (peek().type != TokenType::DOSY) {
            synchronize();
        }
    }

    auto dosy = expect(TokenType::DOSY, NodeType::DoSy, "'do'");
    need(node, dosy);
    if (dosy->isError) synchronize();

    auto stmt = parseStatement();
    if (stmt) {
        need(node, stmt);
    } else {
        SyntaxError err("Diharapkan statement setelah 'do'", peek().line,
peek().column);
        syntaxErrors.push_back(err.what());
        node->addChild(std::make_shared<TreeNode>(NodeType::Error, "<Missing
Statement>", true));
        synchronize();
    }

    return node;
}

// 30 REPEAT-STATEMENT-> repeatsy + STATEMENT-LIST + untilsy + EXPRESSION
std::shared_ptr<TreeNode> Parser::parseRepeatStatement(){
    if (peek().type != TokenType::REPEATSY) return nullptr;

    auto node = std::make_shared<TreeNode>(NodeType::RepeatStatement);
    need(node, terminal(TokenType::REPEATSY, NodeType::RepeatSy));

    auto stmtList = parseStatementList();
    if (stmtList) {
        need(node, stmtList);
    } else {
        SyntaxError err("Diharapkan statement di dalam blok 'repeat'",
peek().line, peek().column);
        syntaxErrors.push_back(err.what());
        node->addChild(std::make_shared<TreeNode>(NodeType::Error, "<Missing
Statement-List>", true));
        synchronize();
    }
}

```

```

    auto untilsy = expect(TokenType::UNTILSY, NodeType::UntilSy, "'until'");
    need(node, untilsy);
    if (untilsy->isError) synchronize();

    auto expr = parseExpression();
    if (expr) {
        need(node, expr);
    } else {
        SyntaxError err("Diharapkan ekspresi kondisi setelah 'until'",
peek().line, peek().column);
        syntaxErrors.push_back(err.what());
        node->addChild(std::make_shared<TreeNode>(NodeType::Error, "<Missing
Expression>", true));
        synchronize();
    }

    return node;
}

// 31 FOR-STATEMENT -> forsy + ident + becomes + EXPRESSION + ( tosy |
downtosy) + EXPRESSION + dosy + STATEMENT
std::shared_ptr<TreeNode> Parser::parseForStatement()
{
    if (peek().type != TokenType::FORSY) {
        return nullptr;
    }

    auto node = std::make_shared<TreeNode>(NodeType::ForStatement);
    need(node, terminal(TokenType::FORSY, NodeType::ForSy));

    auto iden = expect(TokenType::IDENT, NodeType::Ident, "variabel
iterator");
    need(node, iden);
    if (iden->isError) {
        synchronize();
    }

    auto become = expect(TokenType::BECOMES, NodeType::Becomes, "':='");
    need(node, become);
    if (become->isError) {
        synchronize();
    }

    auto startExpression = parseExpression();
    if (startExpression) {
        need(node, startExpression);
    } else {
        SyntaxError err("Diharapkan ekspresi nilai awal setelah '':=''",
peek().line, peek().column);
        syntaxErrors.push_back(err.what());
        node->addChild(std::make_shared<TreeNode>(NodeType::Error, "<Missing

```

```

Start-Expression>", true));
    synchronize();
}

if (peek().type == TokenType::TOSY) {
    need(node, terminal(TokenType::TOSY, NodeType::ToSy));
} else if (peek().type == TokenType::DOWNTOSY) {
    need(node, terminal(TokenType::DOWNTOSY, NodeType::DownToSy));
} else {
    SyntaxError err("Diharapkan 'to' atau 'downto'", peek().line,
peek().column);
    syntaxErrors.push_back(err.what());
    node->addChild(std::make_shared<TreeNode>(NodeType::Error, "<Missing
To/Downto>", true));
}

auto endExpression = parseExpression();
if (endExpression) {
    need(node, endExpression);
} else {
    SyntaxError err("Diharapkan ekspresi batas akhir", peek().line,
peek().column);
    syntaxErrors.push_back(err.what());
    node->addChild(std::make_shared<TreeNode>(NodeType::Error, "<Missing
End-Expression>", true));
    synchronize();
}

auto dosy = expect(TokenType::DOSY, NodeType::DoSy, "'do'");
need(node, dosy);
if (dosy->isError) {
    synchronize();
}

auto statement = parseStatement();
if (statement) {
    need(node, statement);
} else {
    SyntaxError err("Diharapkan statement setelah 'do'", peek().line,
peek().column);
    syntaxErrors.push_back(err.what());
    node->addChild(std::make_shared<TreeNode>(NodeType::Error, "<Missing
Statement>", true));
    synchronize();
}

return node;
}

// 32 PROCEDURE/FUNCTION-CALL ->ident + (lparent + PARAMETER-LIST + rparent)?

```

```

std::shared_ptr<TreeNode> Parser::parseProcedureCall()
{
    if (peek().type != TokenType::IDENT || peek(1).type !=
TokenType::LPARENT){
        return nullptr;
    }

    auto node = std::make_shared<TreeNode>(NodeType::ProcedureCall);
    auto iden = terminal(TokenType::IDENT, NodeType::Ident);
    need(node, iden);
    if (peek().type == TokenType::LPARENT)
    {
        auto lpar = terminal(TokenType::LPARENT, NodeType::LParent);
        need(node, lpar);

        auto parameterList = parseParameterList();
        if (parameterList)
        {
            need(node, parameterList);
        }else{
            SyntaxError err("Diharapkan parameter list", peek().line,
peek().column);
            syntaxErrors.push_back(err.what());
            node->addChild(std::make_shared<TreeNode>(NodeType::Error,
"<Missing Parameter List>", true));
            synchronize();
        }

        auto rpar = expect(TokenType::RPARENT, NodeType::RParent, "");
        need(node, rpar);
        if (rpar->isError){
            synchronize();
        }
    }

    return node;
}

// 33 PARAMETER-LIST -> EXPRESSION (comma + EXPRESSION)*
std::shared_ptr<TreeNode> Parser::parseParameterList()
{
    auto node = std::make_shared<TreeNode>(NodeType::ParameterList);
    auto firstExpression = parseExpression();
    if (firstExpression)
    {
        need(node, firstExpression);
    }else{
        SyntaxError err("Diharapkan sebuah Expression", peek().line,
peek().column);
        syntaxErrors.push_back(err.what());
    }
}

```

```

        node->addChild(std::make_shared<TreeNode>(NodeType::Error, "<Missing
Expression>", true));
        synchronize();
    }

    while (match(TokenType::COMMA)) {
        auto coma = terminal(TokenType::COMMA, NodeType::Comma);
        need(node, coma);
        if (coma->isError){
            synchronize();
        }
        auto nextExpression = parseExpression();
        if (nextExpression){
            need(node, nextExpression);
        }else{
            SyntaxError err("Diharapkan sebuah Expression setelah ','",
peek().line, peek().column);
            syntaxErrors.push_back(err.what());
            node->addChild(std::make_shared<TreeNode>(NodeType::Error,
"<Missing Expression>", true));
            synchronize();
        }
    }

    return node;
}

// 34 EXPRESSION -> SIMPLE-EXPRESSION (RELATIONAL-OPERATOR +
SIMPLE-EXPRESSION)?
std::shared_ptr<TreeNode> Parser::parseExpression()
{
    auto simpleExpr = parseSimpleExpression();
    if (!simpleExpr) return nullptr;

    auto node = std::make_shared<TreeNode>(NodeType::Expression);
    need(node, simpleExpr);

    // Operator Relasional Opsional (Biasanya hanya 1 kali, tidak diulang)
    auto relOp = parseRelationalOperator();
    if (relOp) {
        need(node, relOp);

        auto rightExpr = parseSimpleExpression();
        if (rightExpr) {
            need(node, rightExpr);
        } else {
            SyntaxError err("Diharapkan ekspresi pembanding setelah operator
relasional", peek().line, peek().column);
            syntaxErrors.push_back(err.what());
            node->addChild(std::make_shared<TreeNode>(NodeType::Error,
"<Missing SimpleExpression>", true));

```

```

        synchronize();
    }
}

return node;
}

// 35 SIMPLE-EXPRESSION -> (plus | minus)? TERM (ADDITIVE-OPERATOR + TERM)*
std::shared_ptr<TreeNode> Parser::parseSimpleExpression()
{
    auto node = std::make_shared<TreeNode>(NodeType::SimpleExpression);

    // Opsional + atau - di awal (Unary)
    if (peek().type == TokenType::PLUS)        need(node,
terminal(TokenType::PLUS, NodeType::Plus));
    else if (peek().type == TokenType::MINUS) need(node,
terminal(TokenType::MINUS, NodeType::Minus));

    auto term = parseTerm();
    if (!term) return nullptr; // Jika tidak ada awalan, batalkan
    need(node, term);

    // Selama token berikutnya adalah Additive Operator (+, -, or)
    auto addOp = parseAdditiveOperator();
    while (addOp) {
        need(node, addOp);

        auto nextTerm = parseTerm();
        if (nextTerm) {
            need(node, nextTerm);
        } else {
            SyntaxError err("Diharapkan operand setelah operator penambahan",
peek().line, peek().column);
            syntaxErrors.push_back(err.what());
            node->addChild(std::make_shared<TreeNode>(NodeType::Error,
"<Missing Term>", true));
            synchronize();
        }
        addOp = parseAdditiveOperator();
    }

    return node;
}

// 36 TERM -> FACTOR (MULTIPLICATIVE-OPERATOR + FACTOR)*
std::shared_ptr<TreeNode> Parser::parseTerm()
{
    auto factor = parseFactor();
    if (!factor) return nullptr; // Jika Factor gagal, Term juga gagal (FIRST
Set)

```

```

    auto node = std::make_shared<TreeNode>(NodeType::Term);
    need(node, factor);

    // Selama token berikutnya adalah Multiplicative Operator (*, /, mod,
dll)
    auto multOp = parseMultiplicativeOperator();
    while (multOp) {
        need(node, multOp);

        auto nextFactor = parseFactor();
        if (nextFactor) {
            need(node, nextFactor);
        } else {
            SyntaxError err("Diharapkan operand setelah operator perkalian",
peek().line, peek().column);
            syntaxErrors.push_back(err.what());
            node->addChild(std::make_shared<TreeNode>(NodeType::Error,
"<Missing Factor>", true));
            synchronize();
        }
        multOp = parseMultiplicativeOperator();
    }

    return node;
}

// 37 FACTOR -> (minus)? FACTOR | intcon | charcon | string | (lparent +
EXPRESSION + rparent) | notsy + FACTOR | PROCEDURE/FUNCTION-CALL | VARIABLE
std::shared_ptr<TreeNode> Parser::parseFactor()
{
    TokenType t = peek().type;
    auto node = std::make_shared<TreeNode>(NodeType::Factor);

    // Cek ada minus di depan (Unary Minus)
    if (t == TokenType::MINUS) {
        need(node, terminal(TokenType::MINUS, NodeType::Minus));

        auto innerFactor = parseFactor();
        if (innerFactor) {
            need(node, innerFactor);
        } else {
            SyntaxError err("Diharapkan faktor setelah '-', peek().line,
peek().column);
            syntaxErrors.push_back(err.what());
            node->addChild(std::make_shared<TreeNode>(NodeType::Error,
"<Missing Factor>", true));
            synchronize();
        }
        return node;
    }
}

```



```

// Cek Const
if (t == TokenType::INTCON) { need(node, terminal(t,
NodeType::IntCon)); return node; }
if (t == TokenType::REALCON) { need(node, terminal(t,
NodeType::RealCon)); return node; }
if (t == TokenType::CHARCON) { need(node, terminal(t,
NodeType::CharCon)); return node; }
if (t == TokenType::STRINGCON){ need(node, terminal(t,
NodeType::String)); return node; }

// Cek (lparent + expression + rparent)
if (t == TokenType::LPARENT) {
    need(node, terminal(TokenType::LPARENT, NodeType::LParent));

    auto expr = parseExpression();
    if (expr) need(node, expr);
    else {
        SyntaxError err("Diharapkan ekspresi di dalam kurung",
peek().line, peek().column);
        syntaxErrors.push_back(err.what());
        node->addChild(std::make_shared<TreeNode>(NodeType::Error,
"<Missing Expression>", true));
        synchronize();
    }

    auto rparent = expect(TokenType::RPARENT, NodeType::RParent, "')'");
    need(node, rparent);
    if (rparent->isError) synchronize();
    return node;
}

// Cek (notsy + factor)
if (t == TokenType::NOTSY) {
    need(node, terminal(TokenType::NOTSY, NodeType::NotSy));

    auto innerFactor = parseFactor();
    if (innerFactor) need(node, innerFactor);
    else {
        SyntaxError err("Diharapkan faktor setelah 'not'", peek().line,
peek().column);
        syntaxErrors.push_back(err.what());
        node->addChild(std::make_shared<TreeNode>(NodeType::Error,
"<Missing Factor>", true));
        synchronize();
    }
    return node;
}

// Cek Identifier: Procedure/Function Call ATAU Variable
// Karena keduanya diawali IDENT, ini SATU-SATUNYA tempat kita butuh
save() & restore()

```

```

        if (t == TokenType::IDENT) {
            size_t saved = save();

            // Cek Procedure Call dulu
            auto callNode = parseProcedureCall();
            if (callNode) {
                need(node, callNode);
                return node;
            }

            restore(saved); // Kalau gagal (bukan procedure call), kembalikan
token

            // Cek Variabel
            auto varNode = parseVariable();
            if (varNode) {
                need(node, varNode);
                return node;
            }

            restore(saved);
        }

        return nullptr;
    }

// 38 RELATIONAL-OPERATOR -> eql | neq | gtr | geq | lss | leq
std::shared_ptr<TreeNode> Parser::parseRelationalOperator()
{
    TokenType t = peek().type;
    auto node = std::make_shared<TreeNode>(NodeType::RelationalOperator);

    if (t == TokenType::EQL) need(node, terminal(t, NodeType::Eql));
    else if (t == TokenType::NEQ) need(node, terminal(t, NodeType::Neq));
    else if (t == TokenType::GTR) need(node, terminal(t, NodeType::Gtr));
    else if (t == TokenType::GEQ) need(node, terminal(t, NodeType::Geq));
    else if (t == TokenType::LSS) need(node, terminal(t, NodeType::Lss));
    else if (t == TokenType::LEQ) need(node, terminal(t, NodeType::Leq));
    else return nullptr;

    return node;
}

// 39 ADDITIVE-OPERATOR -> plus | minus | orsy
std::shared_ptr<TreeNode> Parser::parseAdditiveOperator()
{
    TokenType t = peek().type;
    auto node = std::make_shared<TreeNode>(NodeType::AdditiveOperator);

    if (t == TokenType::PLUS) need(node, terminal(t, NodeType::Plus));
    else if (t == TokenType::MINUS) need(node, terminal(t, NodeType::Minus));

```

```

        else if (t == TokenType::ORSY)    need(node, terminal(t, NodeType::OrSy));
        else return nullptr;

        return node;
    }

// 40 MULTIPLICATIVE-OPERATOR -> times | rdiv | idiv | imod | andsy
std::shared_ptr<TreeNode> Parser::parseMultiplicativeOperator()
{
    TokenType t = peek().type;
    auto node = std::make_shared<TreeNode>(NodeType::MultiplicativeOperator);

    if      (t == TokenType::TIMES) need(node, terminal(t, NodeType::Times));
    else if (t == TokenType::RDIV)  need(node, terminal(t, NodeType::RDiv));
    else if (t == TokenType::IDIV)  need(node, terminal(t, NodeType::IDiv));
    else if (t == TokenType::IMOD)  need(node, terminal(t, NodeType::IMod));
    else if (t == TokenType::ANDSY) need(node, terminal(t, NodeType::AndSy));
    else return nullptr;

    return node;
}

// Revisi
std::shared_ptr<TreeNode> Parser::parseVariable()
{
    size_t saved = save();
    auto node = std::make_shared<TreeNode>(NodeType::Variable);
    if (!need(node, terminal(TokenType::IDENT, NodeType::Ident))) return
fail(saved);
    while (need(node, parseComponentVariable())) {
    }
    return node;
}

std::shared_ptr<TreeNode> Parser::parseIndexList()
{
    size_t saved = save();
    auto node = std::make_shared<TreeNode>(NodeType::IndexList);

    if (!(need(node, terminal(TokenType::INTCON, NodeType::IntCon)) ||
        need(node, terminal(TokenType::CHARCON, NodeType::CharCon)) ||
        need(node, terminal(TokenType::IDENT, NodeType::Ident)))) return
fail(saved);

    while (match(TokenType::COMMA)) {
        need(node, expect(TokenType::COMMA, NodeType::Comma, ","));

        if (!(need(node, terminal(TokenType::INTCON, NodeType::IntCon)) ||
            need(node, terminal(TokenType::CHARCON, NodeType::CharCon)) ||
            need(node, terminal(TokenType::IDENT, NodeType::Ident))))
return fail(saved);
    }
}

```

```

    }

    return node;
}

std::shared_ptr<TreeNode> Parser::parseComponentVariable()
{
    size_t saved = save();
    auto node = std::make_shared<TreeNode>(NodeType::ComponentVariable);

    if (match(TokenType::LBRACK)) {
        if (!need(node, terminal(TokenType::LBRACK, NodeType::LBrack)))
            return fail(saved);
        if (!need(node, parseIndexList())) return fail(saved);
        need(node, expect(TokenType::RBRACK, NodeType::RBrack, "]"));
        return node;
    }

    else if (match(TokenType::PERIOD)) {
        if (!need(node, terminal(TokenType::PERIOD, NodeType::Period)))
            return fail(saved);
        need(node, expect(TokenType::IDENT, NodeType::Ident, "record field
identifier"));
        return node;
    }

    return fail(saved);
}

const Token &Parser::peek(int offset) const{
    if (pos + offset >= tokens.size()) {
        if (!tokens.empty()) {
            return tokens.back();
        }

        static Token safeToken(TokenType::PERIOD, "EOF", 0, 0);
        return safeToken;
    }

    return tokens[pos + offset];
}

const Token &Parser::advance(){
    return tokens[pos++];
}

std::string Parser::printNode(const std::shared_ptr<TreeNode> &node){
    if (node == nullptr) {
        return "";
    }

    std::string label = node->typeString();
    if (!node->value.empty()) {
        label += "(" + node->value + ")";
    }
}

```

```

    }
    return label;
}

void Parser::printParseTree(const std::shared_ptr<TreeNode> &node, const
std::string &prefix = "", bool last = true, bool root = true)
{
    if (node == nullptr) {
        return;
    }

    if (root) {
        std::cout << printNode(node) << '\n';
    }
    else {
        std::cout << prefix << (last ? "└─ " : "├─ ") << printNode(node) <<
'\n';
    }

    std::string childPrefix = prefix;
    if (!root) {
        childPrefix += last ? "    " : "│  ";
    }

    for (size_t i = 0; i < node->children.size(); ++i) {
        printParseTree(node->children[i], childPrefix, i + 1 ==
node->children.size(), false);
    }
}

```

2.5. Pemetaan Grammar ke Fungsi Parser

Dalam Recursive Descent, spesifikasi grammar yang didefinisikan dalam format EBNF (Extended Backus-Naur Form) dipetakan secara langsung menjadi fungsi-fungsi di dalam kelas Parser. Pemetaan ini mengikuti aturan Satu Aturan Non-Terminal = Satu Fungsi sesuai dengan aturan production yang ada di Dasar Teori.

Selain itu, notasi-notasi EBNF dipetakan menjadi sebagai berikut

1. $\rightarrow$ menjadi sebuah fungsi independen dengan tipe kembalian pointer node.
2. $\{...\}^*$ menjadi perulangan while. Selama token atau aturan tersebut cocok, parser akan terus memakannya.

3. {...} + {...} menjadi baris instruksi yang ditulis berurutan dari atas ke bawah di dalam blok fungsi. Jika satu instruksi gagal, keseluruhan aturan biasanya dianggap gagal.
4. {...}? menjadi blok if tunggal tanpa else. Jika gagal dipenuhi, program sekadar mengabaikannya dan lanjut ke baris berikutnya.
5. {...} | {...} menjadi struktur percabangan if - else if (menggunakan backtracking need() atau match()).

2.6. Alur Eksekusi Syntax Analyzer dan Integrasi dengan Lexer

Secara garis besar, alur eksekusi program adalah

1. Program membaca source code bahasa Arion dan memvalidasi keberadaan file.
2. Program menjalankan lexical analyzer terhadap source code dan menghasilkan deretan token.
3. Deretan token yang berasal dari lexical analyzer diberikan ke parser melalui Parser parser(tokens).
4. Parser memulai proses parsing dari start symbol program, memanggil fungsi-fungsi recursive descent untuk setiap non-terminal sesuai grammar, dan membangun parse tree secara bertahap.
5. Jika seluruh token berhasil di-parsing tanpa syntax error, parse tree yang dihasilkan dapat ditampilkan di terminal.
6. Jika terjadi syntax error, parsing akan gagal lalu akan ditampilkan jenis kesalahan dan posisi baris yang terlibat.

Sedangkan, alur eksekusi parser tanpa membahas lexer dan bagian lain sebagai berikut:

1. Parser akan menerima daftar token melalui Parser parser(tokens) di main.cpp
2. Proses parsing dimulai ketika fungsi parseProgram() dipanggil. Fungsi ini sebagai titik awal yang merepresentasikan struktur keseluruhan dari sebuah program bahasa Arion. Root node bertipe Program diinstansiasi di sini.
3. Dari parseProgram(), eksekusi dilakukan dengan Top-Down atau memanggil fungsi-fungsi lain yang lebih spesifik. Contoh, parseProgram() akan memanggil parseCompoundStatement(), yang kemudian memanggil parseStatementList(), lalu

memanggil `parseStatement()`, hingga bermuara di `parseExpression()`. Parser akan menyelesaikan evaluasi pada satu cabang tata bahasa sedalam mungkin sebelum beralih mengevaluasi cabang tata bahasa yang lain.

4. Ketika alur rekursif telah mencapai aturan yang tidak bisa dipecah lagi, Parser bertugas mencocokkan teks kode dengan token yang diharapkan.
 - a. Parser menggunakan `match()`, `terminal()`, atau `expect()` untuk melihat token pada indeks pos saat ini.
 - b. Jika token sesuai dengan aturan, token tersebut dikonsumsi. Kursor pos bergeser maju (`pos++`), dan token ini dibungkus menjadi node daun (leaf node).
 - c. Jika token tidak sesuai, Parser melakukan backtracking atau langsung melempar exception (Syntax Error).
5. Meskipun penelusuran aturan dilakukan secara Top-Down, konstruksi parse tree terjadi secara Bottom-Up.
 - a. Setelah token di tingkat terbawah berhasil dikonsumsi dan menjadi node daun, fungsi tersebut mengembalikan node daun itu ke fungsi pemanggilnya di atas.
 - b. Fungsi induk menerima node daun tersebut dan mengaitkannya sebagai anak menggunakan instruksi `node->addChild(...)`.
 - c. Proses penggabungan node anak ke node induk ini terjadi secara berantai dan terus merambat naik seiring berhentinya pemanggilan rekursif.
6. Proses berlangsung hingga mencapai token terakhir, lalu fungsi `parseProgram()` sebagai titik awal akan mengembalikan hasil parse tree.

3. Pengujian

3.1. Kasus 1

Test case:

```
program Test1;

const
  MaxVal == 100;
  MinVal == -10;
  Grade == 'A';
  Welcome == 'Hello Arion';

type
  IdxRange == 1..100;
  Days == (Mon, Tue, Wed, Thu, Fri);
  ScoreArray == array [1..100] of Integer;

var
  x, y, z: Integer;
  myScore: ScoreArray;
  tempArr: array [Mon..Fri] of Integer;

begin
  x := MaxVal;
  y := MinVal;
  z := x + y;
end.
```

Penjelasan:

Test case ini bertujuan untuk menguji kemampuan parser dalam membaca struktur dasar program, mulai dari deklarasi konstanta, variabel dengan tipe data primitif, hingga eksekusi operasi aritmatika dasar dan pemanggilan prosedur bawaan.

Hasil:

Parse tree berhasil dibuat:

```
<program>
├── <program-header>
│   ├── programsy(program)
│   ├── ident(Test1)
│   └── semicolon(;)
├── <declaration-part>
│   ├── <const-declaration>
│   │   ├── constsy(const)
│   │   ├── ident(MaxVal)
│   │   ├── eql(==)
│   │   ├── <constant>
│   │   │   └── intcon(100)
│   │   ├── semicolon(;)
│   │   ├── ident(MinVal)
│   │   ├── eql(==)
│   │   ├── <constant>
│   │   │   ├── minus(-)
│   │   │   └── intcon(10)
│   │   ├── semicolon(;)
│   │   ├── ident(Grade)
│   │   ├── eql(==)
│   │   ├── <constant>
│   │   │   └── charcon(A)
│   │   ├── semicolon(;)
│   │   ├── ident(Welcome)
│   │   ├── eql(==)
│   │   ├── <constant>
│   │   │   └── string(Hello Arion)
│   │   └── semicolon(;)
│   ├── <type-declaration>
│   │   ├── typesy(type)
│   │   ├── ident(IdxRange)
│   │   ├── eql(==)
│   │   ├── <type>
│   │   │   ├── <range>
│   │   │   │   ├── <constant>
│   │   │   │   │   └── intcon(1)
│   │   │   │   ├── period(.)
│   │   │   │   ├── period(.)
│   │   │   │   └── <constant>
│   │   │   │       └── intcon(100)
│   │   ├── semicolon(;)
│   │   ├── ident(Days)
│   │   ├── eql(==)
│   │   ├── <type>
│   │   │   ├── <enumerated>
│   │   │   │   ├── lparent(())
│   │   │   │   ├── ident(Mon)
│   │   │   │   ├── comma(,)
│   │   │   │   ├── ident(Tue)
│   │   │   │   ├── comma(,)
│   │   │   │   ├── ident(Wed)
│   │   │   │   ├── comma(,)
│   │   │   │   ├── ident(Thu)
│   │   │   │   ├── comma(,)
│   │   │   │   ├── ident(Fri)
│   │   │   │   └── rparent())
│   │   ├── semicolon(;)
│   │   ├── ident(ScoreArray)
│   │   ├── eql(==)
│   │   ├── <type>
│   │   │   ├── <array-type>
│   │   │   │   ├── arraysy(array)
│   │   │   │   ├── lbrack([)
│   │   │   │   ├── <range>
│   │   │   │   │   ├── <constant>
│   │   │   │   │   │   └── intcon(1)
│   │   │   │   │   ├── period(.)
│   │   │   │   │   ├── period(.)
│   │   │   │   │   └── <constant>
│   │   │   │   │       └── intcon(100)
│   │   │   │   ├── rbrack(])
│   │   │   │   ├── ofsy(of)
│   │   │   │   ├── <type>
│   │   │   │   │   └── ident(Integer)
│   │   ├── semicolon(;)
│   └── <var-declaration>
│       ├── varsy(var)
│       ├── <identifier-list>
│       │   ├── ident(x)
│       │   ├── comma(,)
│       │   └── ident(y)
```



winter@LAPTOP-F060S22H: ~



```

├── ident(x)
├── comma(,)
├── ident(y)
├── comma(,)
├── ident(z)
├── colon(:)
├── <type>
│   └── ident(Integer)
├── semicolon(;)
├── <identifier-list>
│   └── ident(myScore)
├── colon(:)
├── <type>
│   └── ident(ScoreArray)
├── semicolon(;)
├── <identifier-list>
│   └── ident(tempArr)
├── colon(:)
├── <type>
│   └── <array-type>
│       ├── arrays(array)
│       ├── lbrack([)
│       └── <range>
│           ├── <constant>
│           │   ├── ident(Mon)
│           │   ├── period(.)
│           │   ├── period(.)
│           │   └── <constant>
│           │       └── ident(Fri)
│           ├── rbrack(])
│           ├── ofsy(of)
│           └── <type>
│               └── ident(Integer)
├── semicolon(;)
├── <compound-statement>
│   ├── beginsy(begin)
│   └── <statement-list>
│       ├── <statement>
│       │   └── <assignment-statement>
│       │       ├── <variable>
│       │       │   └── ident(x)
│       │       ├── becomes(:=)
│       │       └── <expression>
│       │           ├── <simple-expression>
│       │           └── <term>
│       │               └── <factor>
│       │                   └── <variable>
│       │                       └── ident(MaxVal)
│       ├── semicolon(;)
│       ├── <statement>
│       │   └── <assignment-statement>
│       │       ├── <variable>
│       │       │   └── ident(y)
│       │       ├── becomes(:=)
│       │       └── <expression>
│       │           ├── <simple-expression>
│       │           └── <term>
│       │               └── <factor>
│       │                   └── <variable>
│       │                       └── ident(MinVal)
│       ├── semicolon(;)
│       ├── <statement>
│       │   └── <assignment-statement>
│       │       ├── <variable>
│       │       │   └── ident(z)
│       │       ├── becomes(:=)
│       │       └── <expression>
│       │           ├── <simple-expression>
│       │           └── <term>
│       │               ├── <factor>
│       │               │   ├── <variable>
│       │               │   └── ident(x)
│       │               ├── <additive-operator>
│       │               │   └── plus(+)
│       │               └── <term>
│       │                   ├── <factor>
│       │                   │   ├── <variable>
│       │                   │   └── ident(y)
│       │                   └── <factor>
│       │                       └── <variable>
│       │                           └── ident(y)
│       ├── semicolon(;)
│       └── <statement>
│           └── endsy(end)
└── period(.)
```

winter@LAPTOP-F060S22H:~/college/JEE-Tubes-IF2224-2026\$

3.2. Kasus 2

Test case:

```
program Kasus2;

var
  i, batas, level: Integer;
  pilihan: Char;

begin
  batas := 5;

  for i := 1 to batas do
    if (i mod 2 == 0) then
      writeln('Genap')
    else
      writeln('Ganjil');
  end;

  while i < 10 do
    i := i + 1;
  end;

  repeat
    batas := batas - 1;
  until batas == 0;

  case pilihan of
    'A': level := 100;
    'B': level := 50;
    'C': level := 10;
  end;

;
end.
```

Penjelasan:

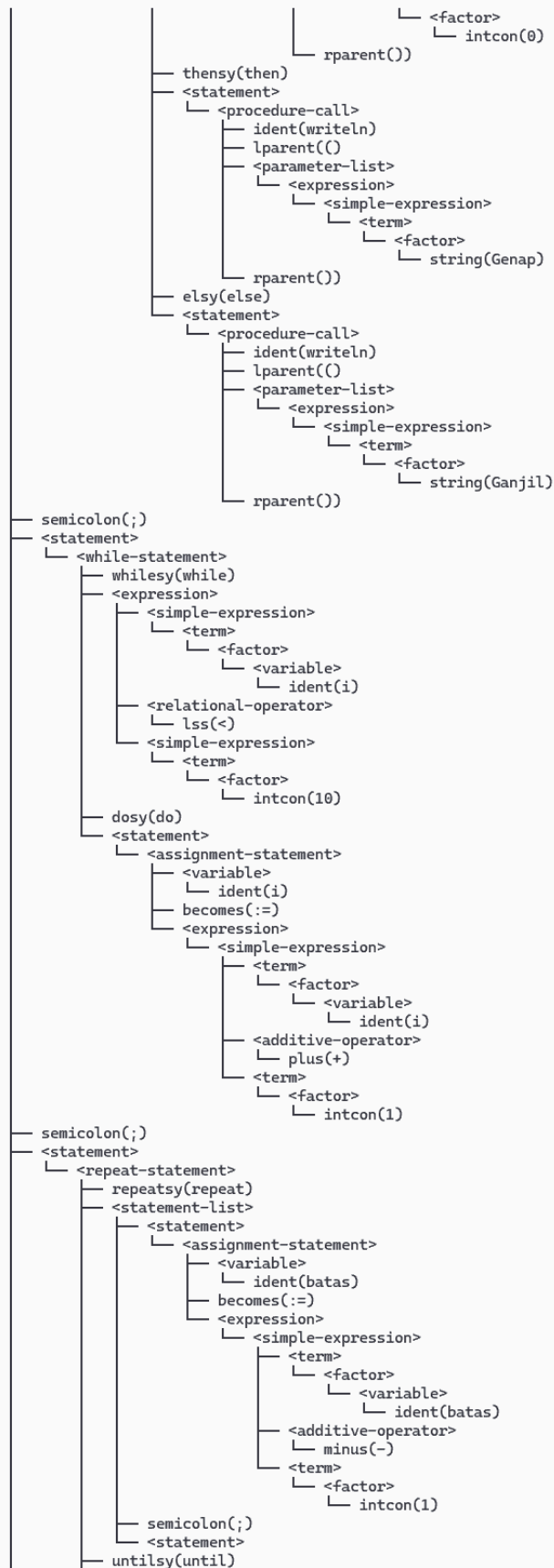
Test case ini bertujuan untuk memvalidasi seluruh control flow (alur kontrol program), baik itu percabangan maupun perulangan, untuk memastikan parser dapat menangani blok logika bersarang dan berbagai jenis kondisi.

Hasil:

```

graph TD
    Root[ ] --- Semicolon1[semicolon(,)]
    Root --- Statement1[<statement>]
    Statement1 --- Until[untilsy(until)]
    Statement1 --- Expression1[<expression>]
    Expression1 --- SimpleExpression1[<simple-expression>]
    SimpleExpression1 --- Term1[<term>]
    Term1 --- Factor1[<factor>]
    Factor1 --- Minus[minus(-)]
    Minus --- AdditiveOperator[<additive-operator>]
    AdditiveOperator --- IdentBatas[ident(batas)]
    SimpleExpression1 --- RelationalOperator[<relational-operator>]
    RelationalOperator --- eql[eql(==)]
    SimpleExpression1 --- SimpleExpression2[<simple-expression>]
    SimpleExpression2 --- Term2[<term>]
    Term2 --- Factor2[<factor>]
    Factor2 --- IdentBatas2[ident(batas)]
    Factor2 --- Intcon1[intcon(1)]
    Statement1 --- Semicolon2[semicolon(,)]
    Statement1 --- Statement2[<statement>]
    Statement2 --- CaseStatement[<case-statement>]
    CaseStatement --- Casesy[casesy(case)]
    Casesy --- Expression2[<expression>]
    Expression2 --- SimpleExpression3[<simple-expression>]
    SimpleExpression3 --- Term3[<term>]
    Term3 --- Factor3[<factor>]
    Factor3 --- Variable1[<variable>]
    Variable1 --- IdentPilihan[ident(pilihan)]
    CaseStatement --- ofsy[ofsy(of)]
    ofsy --- CaseBlock1[<case-block>]
    CaseBlock1 --- Constant1[<constant>]
    Constant1 --- CharconA[charcon(A)]
    CaseBlock1 --- Colon1[colon(:)]
    CaseBlock1 --- Statement3[<statement>]
    Statement3 --- AssignmentStatement[<assignment-statement>]
    AssignmentStatement --- Variable2[<variable>]
    Variable2 --- IdentLevel1[ident(level)]
    AssignmentStatement --- becomes1[becomes(:=)]
    AssignmentStatement --- Expression3[<expression>]
    Expression3 --- SimpleExpression4[<simple-expression>]
    SimpleExpression4 --- Term4[<term>]
    Term4 --- Factor4[<factor>]
    Factor4 --- Intcon100[intcon(100)]
    CaseBlock1 --- Semicolon3[semicolon(,)]
    CaseBlock1 --- CaseBlock2[<case-block>]
    CaseBlock2 --- Constant2[<constant>]
    Constant2 --- CharconB[charcon(B)]
    CaseBlock2 --- Colon2[colon(:)]
    CaseBlock2 --- Statement4[<statement>]
    Statement4 --- AssignmentStatement2[<assignment-statement>]
    AssignmentStatement2 --- Variable3[<variable>]
    Variable3 --- IdentLevel2[ident(level)]
    AssignmentStatement2 --- becomes2[becomes(:=)]
    AssignmentStatement2 --- Expression4[<expression>]
    Expression4 --- SimpleExpression5[<simple-expression>]
    SimpleExpression5 --- Term5[<term>]
    Term5 --- Factor5[<factor>]
    Factor5 --- Intcon50[intcon(50)]
    CaseBlock2 --- Semicolon4[semicolon(,)]
    CaseBlock2 --- CaseBlock3[<case-block>]
    CaseBlock3 --- Constant3[<constant>]
    Constant3 --- CharconC[charcon(C)]
    CaseBlock3 --- Colon3[colon(:)]
    CaseBlock3 --- Statement5[<statement>]
    Statement5 --- AssignmentStatement3[<assignment-statement>]
    AssignmentStatement3 --- Variable4[<variable>]
    Variable4 --- IdentLevel3[ident(level)]
    AssignmentStatement3 --- becomes3[becomes(:=)]
    AssignmentStatement3 --- Expression5[<expression>]
    Expression5 --- SimpleExpression6[<simple-expression>]
    SimpleExpression6 --- Term6[<term>]
    Term6 --- Factor6[<factor>]
    Factor6 --- Intcon10[intcon(10)]
    CaseBlock3 --- endsy[endsy(end)]
    Statement2 --- Semicolon5[semicolon(,)]
    Statement2 --- Statement6[<statement>]
    Statement6 --- Semicolon6[semicolon(,)]
    Statement6 --- Statement7[<statement>]
    Statement7 --- endsy2[endsy(end)]
    Root --- period[period(.)]

```





winter@LAPTOP-F060S22H: ~



=> State s9 => Gotten: period(.)

. => char => State 0 => Gotten: period(.)

Parse tree berhasil dibuat:

```
<program>
├── <program-header>
│   ├── programsy(program)
│   ├── ident(Kasus2)
│   └── semicolon(;)
├── <declaration-part>
│   └── <var-declaration>
│       ├── varsy(var)
│       ├── <identifier-list>
│       │   ├── ident(i)
│       │   ├── comma(,)
│       │   ├── ident(batas)
│       │   ├── comma(,)
│       │   └── ident(level)
│       ├── colon(:)
│       ├── <type>
│       │   └── ident(Integer)
│       ├── semicolon(;)
│       ├── <identifier-list>
│       │   └── ident(pilihan)
│       ├── colon(:)
│       ├── <type>
│       │   └── ident(Char)
│       └── semicolon(;)
└── <compound-statement>
    ├── beginsy(begin)
    └── <statement-list>
        ├── <statement>
        │   ├── <assignment-statement>
        │   │   ├── <variable>
        │   │   │   └── ident(batas)
        │   │   ├── becomes(:=)
        │   │   └── <expression>
        │   │       ├── <simple-expression>
        │   │       │   ├── <term>
        │   │       │   │   └── <factor>
        │   │       │   │       └── intcon(5)
        │   └── semicolon(;)
        ├── <statement>
        │   ├── <for-statement>
        │   │   ├── forsy(for)
        │   │   ├── ident(i)
        │   │   ├── becomes(:=)
        │   │   └── <expression>
        │   │       ├── <simple-expression>
        │   │       │   ├── <term>
        │   │       │   │   └── <factor>
        │   │       │   │       └── intcon(1)
        │   ├── tosy(to)
        │   ├── <expression>
        │   │   ├── <simple-expression>
        │   │   │   ├── <term>
        │   │   │   │   └── <factor>
        │   │   │   │       ├── <variable>
        │   │   │   │       │   └── ident(batas)
        │   ├── dosy(do)
        │   └── <statement>
        │       ├── <if-statement>
        │       │   ├── ifsy(if)
        │       │   └── <expression>
        │       │       ├── <simple-expression>
        │       │       │   ├── <term>
        │       │       │   │   └── <factor>
        │       │       │   │       ├── lparent()
        │       │       │   │       ├── <expression>
        │       │       │   │       │   ├── <simple-expression>
        │       │       │   │       │   │   ├── <term>
        │       │       │   │       │   │   │   ├── <factor>
        │       │       │   │       │   │   │   │   ├── <variable>
        │       │       │   │       │   │   │   │   │   └── ident(i)
        │       │       │   │       │   │   │   ├── <multiplicative-operator>
        │       │       │   │       │   │   │   │   └── imod(mod)
        │       │       │   │       │   │   │   └── <factor>
        │       │       │   │       │   │   │       └── intcon(2)
        │       │       │   │       │   ├── <relational-operator>
        │       │       │   │       │   │   └── eql(==)
        │       │       │   │       └── <simple-expression>
        │       │       │           ├── <term>
        │       │       │           │   └── <factor>
        │       │       │           │       └── intcon(0)
        │       │       └── rparent())
        └── <statement>
```

3.3. Kasus 3

Test case:

```
program Kasus3;

type
  Hari == (Senin, Selasa, Rabu, Kamis, Jumat);
  NilaiUjian == 0..100;

  Siswa == record
    nama: String;
    umur: Integer;
    nilai: array [1..5] of NilaiUjian
  end;

var
  kelas: array [1..30] of Siswa;
  hariIni: Hari;

begin
  hariIni := Senin;
  kelas[1].nama := 'Budi';
  kelas[1].umur := 16;
  kelas[1].nilai[1] := 85;
  kelas[1].nilai[2] := 90
end.
```

Penjelasan:

Test case ini dirancang khusus untuk menguji kemampuan parser dalam mengenali tipe data bentukan pengguna (user-defined types) yang kompleks, serta cara mengakses elemen-elemen di dalam tipe data terstruktur tersebut.

Hasil:

```
winter@LAPTOP-F060S22H:~/college/JEE-Tubes-IF2224-2026$
```




winter@LAPTOP-F060S22H: ~



```
varsy(var)
<identifier-list>
  ident(kelas)
colon(:)
<type>
  <array-type>
    arraysy(array)
    lbrack([])
    <range>
      <constant>
        intcon(1)
      period(.)
      period(.)
      <constant>
        intcon(30)
    rbrack()]
    ofsy(of)
    <type>
      ident(Siswa)
  semicolon(;)
<identifier-list>
  ident(hariIni)
colon(:)
<type>
  ident(Hari)
  semicolon(;)
<compound-statement>
  beginsy(begin)
  <statement-list>
    <statement>
      <assignment-statement>
        <variable>
          ident(hariIni)
        becomes(:=)
        <expression>
          <simple-expression>
            <term>
              <factor>
                <variable>
                  ident(Senin)
      semicolon(;)
    <statement>
      <assignment-statement>
        <variable>
          ident(kelas)
          <component-variable>
            lbrack([])
            <index-list>
              intcon(1)
            rbrack()]
          <component-variable>
            period(.)
            ident(nama)
        becomes(:=)
        <expression>
          <simple-expression>
            <term>
              <factor>
                string(Budi)
      semicolon(;)
    <statement>
      <assignment-statement>
        <variable>
          ident(kelas)
          <component-variable>
            lbrack([])
            <index-list>
              intcon(1)
            rbrack()]
          <component-variable>
            period(.)
            ident(umur)
        becomes(:=)
        <expression>
          <simple-expression>
            <term>
              <factor>
                intcon(16)
      semicolon(;)
    <statement>
      <assignment-statement>
        <variable>
          ident(kelas)
          <component-variable>
```



winter@LAPTOP-F060S22H: ~



```
State 0 => . => State s0
=> State s9 => Gotten: period(.)
. => char => State 0 => Gotten: period(.)
Parse tree berhasil dibuat:
<program>
├── <program-header>
│   ├── programsy(program)
│   ├── ident(Kasus3)
│   └── semicolon(;)
├── <declaration-part>
│   ├── <type-declaration>
│   │   ├── typesy(type)
│   │   ├── ident(Hari)
│   │   ├── eql(==)
│   │   └── <type>
│   │       └── <enumerated>
│   │           ├── lparent()
│   │           ├── ident(Senin)
│   │           ├── comma(,)
│   │           ├── ident(Selasa)
│   │           ├── comma(,)
│   │           ├── ident(Rabu)
│   │           ├── comma(,)
│   │           ├── ident(Kamis)
│   │           ├── comma(,)
│   │           ├── ident(Jumat)
│   │           └── rparent())
│   ├── semicolon(;)
│   ├── ident(NilaiUjian)
│   ├── eql(==)
│   └── <type>
│       ├── <range>
│       │   ├── <constant>
│       │   │   └── intcon(0)
│       │   ├── period(.)
│       │   ├── period(.)
│       │   └── <constant>
│       │       └── intcon(100)
│       ├── semicolon(;)
│       ├── ident(Siswa)
│       ├── eql(==)
│       └── <type>
│           ├── <record-type>
│           │   ├── recordsy(record)
│           │   └── <field-list>
│           │       ├── <field-part>
│           │       │   ├── <identifier-list>
│           │       │   │   └── ident(nama)
│           │       │   ├── colon(:)
│           │       │   └── <type>
│           │       │       └── ident(String)
│           │       ├── semicolon(;)
│           │       ├── <field-part>
│           │       │   ├── <identifier-list>
│           │       │   │   └── ident(umur)
│           │       │   ├── colon(:)
│           │       │   └── <type>
│           │       │       └── ident(Integer)
│           │       ├── semicolon(;)
│           │       ├── <field-part>
│           │       │   ├── <identifier-list>
│           │       │   │   └── ident(nilai)
│           │       │   ├── colon(:)
│           │       │   └── <type>
│           │       │       ├── <array-type>
│           │       │       │   ├── arraysy(array)
│           │       │       │   ├── lbrack([)
│           │       │       │   └── <range>
│           │       │       │       ├── <constant>
│           │       │       │       │   └── intcon(1)
│           │       │       │       ├── period(.)
│           │       │       │       ├── period(.)
│           │       │       │       └── <constant>
│           │       │       │           └── intcon(5)
│           │       │       ├── rbrack(])
│           │       │       ├── ofsy(of)
│           │       │       └── <type>
│           │       │           └── ident(NilaiUjian)
│           │       └── endsy(end)
│           ├── semicolon(;)
│           └── <var-declaration>
│               ├── varsy(var)
│               └── <identifier-list>
│                   └── ident(kelas)
└── endsy(end)
```

3.4. Kasus 4

Test case:

```
program Kasus4;

var
    hasil: Integer;

procedure Tukar(a, b: Integer; mode: Char);
var
    temp: Integer;
begin
    if mode == 'X' then
        temp := a
    end;

function Faktorial(n: Integer): Integer;
begin
    if n <= 1 then
        Faktorial := 1
    else
        Faktorial := n * Faktorial(n - 1);
    end;

begin
    Tukar(10, 20, 'X');
    hasil := Faktorial(5);
end.
```

Penjelasan:

Test case ini dirancang khusus untuk menguji kemampuan parser dalam mengenali tipe data bentukan pengguna (user-defined types) yang kompleks, serta cara mengakses elemen-elemen di dalam tipe data terstruktur tersebut.

Hasil:



winter@LAPTOP-F060S22H: ~



```

└─ <assignment-statement>
  └─ <variable>
    └─ ident(Faktorial)
  └─ becomes(:=)
  └─ <expression>
    └─ <simple-expression>
      └─ <term>
        └─ <factor>
          └─ <variable>
            └─ ident(n)
        └─ <multiplicative-operator>
          └─ times(*)
        └─ <factor>
          └─ <procedure-call>
            └─ ident(Faktorial)
            └─ lparent()
            └─ <parameter-list>
              └─ <expression>
                └─ <simple-expression>
                  └─ <term>
                    └─ <factor>
                      └─ <variable>
                        └─ ident(n)
                    └─ <additive-operator>
                      └─ minus(-)
                    └─ <term>
                      └─ <factor>
                        └─ intcon(1)
            └─ rparent()
          └─ semicolon(;)
        └─ <statement>
          └─ endsy(end)
        └─ semicolon(;)
      └─ <compound-statement>
        └─ beginsy(begin)
        └─ <statement-list>
          └─ <statement>
            └─ <procedure-call>
              └─ ident(Tukar)
              └─ lparent()
              └─ <parameter-list>
                └─ <expression>
                  └─ <simple-expression>
                    └─ <term>
                      └─ <factor>
                        └─ intcon(10)
                └─ comma(,)
                └─ <expression>
                  └─ <simple-expression>
                    └─ <term>
                      └─ <factor>
                        └─ intcon(20)
                └─ comma(,)
                └─ <expression>
                  └─ <simple-expression>
                    └─ <term>
                      └─ <factor>
                        └─ charcon(X)
              └─ rparent()
            └─ semicolon(;)
          └─ <statement>
            └─ <assignment-statement>
              └─ <variable>
                └─ ident(hasil)
              └─ becomes(:=)
              └─ <expression>
                └─ <simple-expression>
                  └─ <term>
                    └─ <factor>
                      └─ <procedure-call>
                        └─ ident(Faktorial)
                        └─ lparent()
                        └─ <parameter-list>
                          └─ <expression>
                            └─ <simple-expression>
                              └─ <term>
                                └─ <factor>
                                  └─ intcon(5)
                        └─ rparent()
                      └─ semicolon(;)
                    └─ <statement>
                      └─ endsy(end)
                    └─ period(.)
          └─ semicolon(;)
        └─ endsy(end)
      └─ period(.)
    └─ winter@LAPTOP-F060S22H:~/college/JEE-Tubes-IF2224-2026$
```



winter@LAPTOP-F060S22H: ~



```

├── ident(Faktorial)
├── <formal-parameter-list>
│   ├── lparent()
│   ├── <parameter-group>
│   │   ├── <identifier-list>
│   │   │   └── ident(n)
│   │   ├── colon(:)
│   │   └── ident(Integer)
│   └── rparent()
├── colon(:)
├── ident(Integer)
├── semicolon(;)
├── block
│   ├── <declaration-part>
│   ├── <compound-statement>
│   │   ├── beginsy(begin)
│   │   ├── <statement-list>
│   │   │   ├── <statement>
│   │   │   │   ├── <if-statement>
│   │   │   │   │   ├── ifsy(if)
│   │   │   │   │   │   ├── <expression>
│   │   │   │   │   │   │   ├── <simple-expression>
│   │   │   │   │   │   │   │   ├── <term>
│   │   │   │   │   │   │   │   │   ├── <factor>
│   │   │   │   │   │   │   │   │   │   ├── <variable>
│   │   │   │   │   │   │   │   │   │   │   └── ident(n)
│   │   │   │   │   │   │   │   ├── <relational-operator>
│   │   │   │   │   │   │   │   │   └── leq(<=)
│   │   │   │   │   │   │   └── <simple-expression>
│   │   │   │   │   │   │       ├── <term>
│   │   │   │   │   │   │       │   ├── <factor>
│   │   │   │   │   │   │       │   └── intcon(1)
│   │   │   │   │   ├── thensy(then)
│   │   │   │   │   │   ├── <statement>
│   │   │   │   │   │   │   ├── <assignment-statement>
│   │   │   │   │   │   │   │   ├── <variable>
│   │   │   │   │   │   │   │   │   └── ident(Faktorial)
│   │   │   │   │   │   │   ├── becomes(:=)
│   │   │   │   │   │   │   └── <expression>
│   │   │   │   │   │   │       ├── <simple-expression>
│   │   │   │   │   │   │       │   ├── <term>
│   │   │   │   │   │   │       │   │   ├── <factor>
│   │   │   │   │   │   │       │   │   └── intcon(1)
│   │   │   │   │   │   └── elsy(else)
│   │   │   │   │   │       ├── <statement>
│   │   │   │   │   │       │   ├── <assignment-statement>
│   │   │   │   │   │       │   │   ├── <variable>
│   │   │   │   │   │       │   │   │   └── ident(Faktorial)
│   │   │   │   │   │       │   ├── becomes(:=)
│   │   │   │   │   │       │   └── <expression>
│   │   │   │   │   │       │       ├── <simple-expression>
│   │   │   │   │   │       │       │   ├── <term>
│   │   │   │   │   │       │       │   │   ├── <factor>
│   │   │   │   │   │       │       │   │   │   ├── <variable>
│   │   │   │   │   │       │       │   │   │   └── ident(n)
│   │   │   │   │   │       │       │   ├── <multiplicative-operator>
│   │   │   │   │   │       │       │   │   └── times(*)
│   │   │   │   │   │       │       └── <factor>
│   │   │   │   │   │       │           ├── <procedure-call>
│   │   │   │   │   │       │           │   ├── ident(Faktorial)
│   │   │   │   │   │       │           │   ├── lparent()
│   │   │   │   │   │       │           │   ├── <parameter-list>
│   │   │   │   │   │       │           │   │   ├── <expression>
│   │   │   │   │   │       │           │   │   │   ├── <simple-expression>
│   │   │   │   │   │       │           │   │   │   │   ├── <term>
│   │   │   │   │   │       │           │   │   │   │   │   ├── <factor>
│   │   │   │   │   │       │           │   │   │   │   │   │   ├── <variable>
│   │   │   │   │   │       │           │   │   │   │   │   │   └── ident(n)
│   │   │   │   │   │       │           │   │   │   │   ├── <additive-operator>
│   │   │   │   │   │       │           │   │   │   │   │   └── minus(-)
│   │   │   │   │   │       │           │   │   │   └── <term>
│   │   │   │   │   │       │           │   │   │       ├── <factor>
│   │   │   │   │   │       │           │   │   │       └── intcon(1)
│   │   │   │   │   │       │           └── rparent()
│   │   │   │   │   └── semicolon(;)
│   │   │   │   │   └── <statement>
│   │   │   │   └── endsy(end)
│   │   └── semicolon(;)
│   └── <compound-statement>
│       ├── beginsy(begin)
│       ├── <statement-list>
│       │   ├── <statement>
│       │   │   ├── <procedure-call>
│       │   │   │   └── ident(Tukar)

```



winter@LAPTOP-F060S22H: ~



```
=> State s9 => Gotten: period(.)
. => char => State 0 => Gotten: period(.)
Parse tree berhasil dibuat:
<program>
├── <program-header>
│   ├── programsy(program)
│   ├── ident(Kasus4)
│   └── semicolon(;)
├── <declaration-part>
│   ├── <var-declaration>
│   │   ├── varsy(var)
│   │   ├── <identifier-list>
│   │   │   └── ident(hasil)
│   │   ├── colon(:)
│   │   ├── <type>
│   │   │   └── ident(Integer)
│   │   └── semicolon(;)
│   ├── <procedure-declaration>
│   │   ├── proceduresy(procedure)
│   │   ├── ident(Tukar)
│   │   ├── <formal-parameter-list>
│   │   │   ├── lparent()
│   │   │   ├── <parameter-group>
│   │   │   │   ├── <identifier-list>
│   │   │   │   │   ├── ident(a)
│   │   │   │   │   ├── comma(,)
│   │   │   │   │   └── ident(b)
│   │   │   │   ├── colon(:)
│   │   │   │   └── ident(Integer)
│   │   │   └── semicolon(;)
│   │   │   └── <parameter-group>
│   │   │       ├── <identifier-list>
│   │   │       │   └── ident(mode)
│   │   │       ├── colon(:)
│   │   │       └── ident(Char)
│   │   │       └── rparent())
│   │   └── semicolon(;)
│   └── block
│       ├── <declaration-part>
│       │   ├── <var-declaration>
│       │   │   ├── varsy(var)
│       │   │   ├── <identifier-list>
│       │   │   │   └── ident(temp)
│       │   │   ├── colon(:)
│       │   │   ├── <type>
│       │   │   │   └── ident(Integer)
│       │   │   └── semicolon(;)
│       │   └── <compound-statement>
│       │       ├── beginsy(begin)
│       │       ├── <statement-list>
│       │       │   └── <statement>
│       │       │       ├── <if-statement>
│       │       │       │   ├── ifsy(if)
│       │       │       │   ├── <expression>
│       │       │       │   │   ├── <simple-expression>
│       │       │       │   │   │   ├── <term>
│       │       │       │   │   │   │   ├── <factor>
│       │       │       │   │   │   │   │   ├── <variable>
│       │       │       │   │   │   │   │   │   └── ident(mode)
│       │       │       │   │   │   │   └── <relational-operator>
│       │       │       │   │   │   │   │   └── eql(==)
│       │       │       │   │   │   └── <simple-expression>
│       │       │       │   │   │       ├── <term>
│       │       │       │   │   │       │   ├── <factor>
│       │       │       │   │   │       │   │   └── charcon(X)
│       │       │       │   └── thensy(then)
│       │       │       │       ├── <statement>
│       │       │       │       │   ├── <assignment-statement>
│       │       │       │       │   │   ├── <variable>
│       │       │       │       │   │   │   └── ident(temp)
│       │       │       │       │   ├── becomes(:=)
│       │       │       │       │   ├── <expression>
│       │       │       │       │   │   ├── <simple-expression>
│       │       │       │       │   │   │   ├── <term>
│       │       │       │       │   │   │   │   ├── <factor>
│       │       │       │       │   │   │   │   │   ├── <variable>
│       │       │       │       │   │   │   │   │   │   └── ident(a)
│       │       │       │       └── endsy(end)
│       │       └── semicolon(;)
│       └── <function-declaration>
│           ├── functionsy(function)
│           ├── ident(Faktorial)
│           ├── <formal-parameter-list>
│           │   └── lparent()
└── semicolon(;)
```

3.5. Kasus 5

Test case:

```
program Kasus5;

var
  a, b, c: Integer;
  valid: Boolean;

begin
  a := 10;
  b := 3;

  c := a div b + a mod b * 5 - (a + 2) * (b - 1);

  valid := (a > b) AND (c <> 0) OR NOT (a <= 5);

  if valid then
    c := c * 1;
end.
```

Penjelasan:

Test case ini bertujuan untuk mengevaluasi deklarasi subprogram, termasuk ruang lingkup (scope) variabel lokal, pengiriman parameter aktual dan formal, serta pemanggilan subprogram di dalam blok utama maupun secara rekursif.

Hasil:



winter@LAPTOP-F060S22H: ~



State 0 => . => State s0
=> State s9 => Gotten: period(.)
. => char => State 0 => Gotten: period(.)
Parse tree berhasil dibuat:

```
<program>
├── <program-header>
│   ├── programsy(program)
│   ├── ident(Kasus5)
│   └── semicolon(;)
├── <declaration-part>
│   ├── <var-declaration>
│   │   ├── varsy(var)
│   │   ├── <identifier-list>
│   │   │   ├── ident(a)
│   │   │   ├── comma(,)
│   │   │   ├── ident(b)
│   │   │   ├── comma(,)
│   │   │   └── ident(c)
│   │   ├── colon(:)
│   │   ├── <type>
│   │   │   └── ident(Integer)
│   │   ├── semicolon(;)
│   │   ├── <identifier-list>
│   │   │   └── ident(valid)
│   │   ├── colon(:)
│   │   ├── <type>
│   │   │   └── ident(Boolean)
│   │   └── semicolon(;)
│   └── <compound-statement>
│       ├── beginsy(begin)
│       └── <statement-list>
│           ├── <statement>
│           │   ├── <assignment-statement>
│           │   │   ├── <variable>
│           │   │   │   └── ident(a)
│           │   │   ├── becomes(:=)
│           │   │   └── <expression>
│           │   │       ├── <simple-expression>
│           │   │       │   └── <term>
│           │   │       │       └── <factor>
│           │   │       │           └── intcon(10)
│           │   └── semicolon(;)
│           ├── <statement>
│           │   ├── <assignment-statement>
│           │   │   ├── <variable>
│           │   │   │   └── ident(b)
│           │   │   ├── becomes(:=)
│           │   │   └── <expression>
│           │   │       ├── <simple-expression>
│           │   │       │   └── <term>
│           │   │       │       └── <factor>
│           │   │       │           └── intcon(3)
│           │   └── semicolon(;)
│           └── <statement>
│               ├── <assignment-statement>
│               │   ├── <variable>
│               │   │   └── ident(c)
│               │   ├── becomes(:=)
│               │   └── <expression>
│               │       ├── <simple-expression>
│               │       │   ├── <term>
│               │       │   │   ├── <factor>
│               │       │   │   │   ├── <variable>
│               │       │   │   │   │   └── ident(a)
│               │       │   │   │   ├── <multiplicative-operator>
│               │       │   │   │   │   ├── idiv(div)
│               │       │   │   │   │   └── <factor>
│               │       │   │   │   │   ├── <variable>
│               │       │   │   │   │   │   └── ident(b)
│               │       │   │   │   └── <additive-operator>
│               │       │   │   │       └── plus(+)
│               │       │   │   └── <term>
│               │       │   │       ├── <factor>
│               │       │   │       │   ├── <variable>
│               │       │   │       │   │   └── ident(a)
│               │       │   │       │   ├── <multiplicative-operator>
│               │       │   │       │   │   ├── imod(mod)
│               │       │   │       │   │   └── <factor>
│               │       │   │       │   │   ├── <variable>
│               │       │   │       │   │   │   └── ident(b)
│               │       │   │       │   └── <multiplicative-operator>
│               │       │   │       │       ├── times(*)
│               │       │   │       │       └── <factor>
│               │       │   │       │           └── intcon(5)
│               │       │   └── <term>
│               │       │       └── <factor>
│               │       │           └── intcon(5)
```




winter@LAPTOP-F060S22H: ~



```

├── times(*)
├── <factor>
├── intcon(5)
├── <additive-operator>
├── minus(-)
├── <term>
├── <factor>
├── lparent()
├── <expression>
├── <simple-expression>
├── <term>
├── <factor>
├── <variable>
├── ident(a)
├── <additive-operator>
├── plus(+)
├── <term>
├── <factor>
├── intcon(2)
├── rparent())
├── <multiplicative-operator>
├── times(*)
├── <factor>
├── lparent()
├── <expression>
├── <simple-expression>
├── <term>
├── <factor>
├── <variable>
├── ident(b)
├── <additive-operator>
├── minus(-)
├── <term>
├── <factor>
├── intcon(1)
├── rparent())
├── semicolon(;)
├── <statement>
├── <assignment-statement>
├── <variable>
├── ident(valid)
├── becomes(:=)
├── <expression>
├── <simple-expression>
├── <term>
├── <factor>
├── lparent()
├── <expression>
├── <simple-expression>
├── <term>
├── <factor>
├── <variable>
├── ident(a)
├── <relational-operator>
├── gtr(>)
├── <simple-expression>
├── <term>
├── <factor>
├── <variable>
├── ident(b)
├── rparent())
├── <multiplicative-operator>
├── andsy(AND)
├── <factor>
├── lparent()
├── <expression>
├── <simple-expression>
├── <term>
├── <factor>
├── <variable>
├── ident(c)
├── <relational-operator>
├── neq(<=)
├── <simple-expression>
├── <term>
├── <factor>
├── intcon(0)
├── rparent())
├── <additive-operator>
├── orsy(OR)
├── <term>
├── <factor>
├── notsy(NOT)
├── <factor>

```



3.6. Kasus 6

Test case:

```
program Kasus6;

const
  Limit = 10;

var
  a, b, c: Integer;
  valid: Boolean;

begin
  a := 10;
  b := 3;
  valid := a > b;

  while a < Limit do
  begin
    if valid then
      c := a + b
      a := a + 1;
    end;
  end.
end.
```

Penjelasan:

Test case ini bertujuan untuk mengevaluasi ketahanan *parser* terhadap kesalahan penggunaan operator relasional pada blok deklarasi konstanta. Pengujian ini memastikan parser menolak penggunaan operator assignment (:=) secara ketat dan menagih operator equality (==) sesuai dengan spesifikasi terminal pada aturan <const-declaration>.

Hasil:

```
Syntax errors found:
- [Syntax Error] at Line 4, Col 6: Diharapkan '==' tetapi menemukan ':= '
- [Syntax Error] at Line 4, Col 6: Diharapkan nilai konstanta
- [Syntax Error] at Line 6, Col 1: Diharapkan ';' tetapi menemukan 'var'
make: *** [makefile:23: run] Error 1
winter@LAPTOP-F060S22H:~/college/JEE-Tubes-IF2224-2026$ |
```

3.7. Kasus 7

Test case:

```
program UjiErrorDua;

var
  i: Integer;

begin
```

```

i := 0;

while i < 5 do
begin
  i := i + 1;
end;
end.

```

Penjelasan:

Test case ini bertujuan untuk mengevaluasi ketatnya pembatasan struktural pada pernyataan kendali (control flow). Pengujian ini memastikan mekanisme fail-fast pada parser berjalan ketika menemukan pembungkus blok begin..end yang ilegal di dalam <while-statement>, mengingat tata bahasa secara eksplisit hanya mengizinkan eksekusi satu <statement> tunggal tanpa compound statement.

Hasil:

```

Syntax errors found:
- [Syntax Error] at Line 10, Col 3: Diharapkan 'end' tetapi menemukan 'begin'
- [Syntax Error] at Line 12, Col 3: Diharapkan . tetapi menemukan 'end'
make: *** [makefile:23: run] Error 1
winter@LAPTOP-F060S22H:~/college/JEE-Tubes-IF2224-2026$ |

```

3.8. Kasus 8

Test case:

```

program UjiErrorTiga;

type
  Siswa == record
    nama: String;
    umur: Integer;
  end;

var
  ketua: Siswa;

begin
  ketua.umur := 17
end.

```

Penjelasan:

Test case ini bertujuan untuk mengevaluasi presisi pemetaan perulangan berurutan (while loop) pada aturan <record-type>. Pengujian ini memastikan parser mendeteksi dan menggagalkan kompilasi akibat keberadaan karakter titik koma (;) yang berlebih

(trailing semicolon) pada akhir <field-list> sebelum token penutup end, yang menyalahi kaidah sekuensial tata bahasa.

Hasil:

```
Syntax errors found:
- [Syntax Error] at Line 7, Col 3: Diharapkan identifier tetapi menemukan 'end'
- [Syntax Error] at Line 7, Col 3: Diharapkan : tetapi menemukan 'end'
- [Syntax Error] at Line 9, Col 1: Diharapkan Type Data
- [Syntax Error] at Line 12, Col 1: Diharapkan end tetapi menemukan 'begin'
- [Syntax Error] at Line 14, Col 5: Diharapkan ';' tetapi menemukan 'EOF'
- [Syntax Error] at Line 14, Col 5: Diharapkan . tetapi menemukan 'EOF'
make: *** [makefile:23: run] Error 1
winter@LAPTOP-F060S22H:~/college/JEE-Tubes-IF2224-2026$ |
```

3.9. Kasus 9

Test case:

```
program Kasus9;

var
  a, b: Integer;

begin
  a := 10;
  b := 3;

  case a + b of
    1, 2
      a := a + 1;
    3: b := b + 1;
  end;
end.
```

Penjelasan:

Test case ini bertujuan untuk mengevaluasi kelengkapan komponen wajib (mandatory tokens) pada aturan <function-declaration>. Pengujian ini menguji kemampuan fungsi expect() di dalam parser untuk langsung melempar Syntax Error ketika sebuah definisi subprogram fungsi tidak menyertakan deklarasi tipe data pengembalian (return type) setelah penutup parameter formal.

Hasil:

```
Syntax errors found:
- [Syntax Error] at Line 6, Col 31: Diharapkan ':' tetapi menemukan ';'
- [Syntax Error] at Line 7, Col 1: Diharapkan tipe kembalian function tetapi menemukan 'begin'
- [Syntax Error] at Line 11, Col 1: Diharapkan ';' tetapi menemukan 'begin'
- [Syntax Error] at Line 0, Col 0: Diharapkan blok 'begin ... end' (Compound Statement)
- [Syntax Error] at Line 0, Col 0: Diharapkan ';' di akhir function tetapi menemukan ''
- [Syntax Error] at Line 0, Col 0: Diharapkan . tetapi menemukan ''
make: *** [makefile:23: run] Error 1
winter@LAPTOP-F060S22H:~/college/JEE-Tubes-IF2224-2026$ |
```

3.10. Kasus 10

Test case:

```
program UjiErrorLima;

var
  a, b, c: Integer;

begin
  a := 10;
  b := 5;
  c := (a + b * 2;
end.
```

Penjelasan:

Test case ini bertujuan untuk mengevaluasi keandalan mekanisme resolusi ekspresi hierarkis dan penanganan operator pengelompokan di tingkat paling dasar (<factor>). Pengujian ini memastikan parser secara ketat menagih keberadaan token kurung tutup (RPARENT) untuk setiap kurung buka, dan menghindari evaluasi Parse Tree parsial jika ekspresi matematika tidak tertutup dengan benar.

Hasil:

```
Syntax errors found:
- [Syntax Error] at Line 9, Col 18: Diharapkan ')' tetapi menemukan ';'
make: *** [makefile:23: run] Error 1
winter@LAPTOP-F060S22H:~/college/JEE-Tubes-IF2224-2026$ |
```

4. Kesimpulan dan Saran

4.1. Kesimpulan

Pada Milestone 2 tugas besar IF2224 ini, kami mengimplementasikan syntax analyzer untuk bahasa Arion menggunakan metode Recursive Descent. Prosesnya dimulai dengan merancang context-free grammar dan notasi EBNF untuk mengatur struktur deklarasi, statement, dan expression, lengkap dengan aturan presedensi operator. Setiap aturan non-terminal kemudian diterjemahkan ke dalam fungsi-fungsi parsing rekursif yang secara otomatis menyusun parse tree secara hierarkis.

Parser ini memproses rangkaian token dari lexer Milestone 1 untuk memastikan validitas sintaksis dalam program bahasa Arion sehingga sistem dapat mengenali berbagai elemen dalam bahasa Arion. Kami juga menyertakan fitur error handling untuk mendeteksi kesalahan seperti kekurangan titik koma atau kesalahan kata kunci.

Melalui pengujian pada berbagai skrip, parser ini terbukti mampu menghasilkan parse tree yang sesuai dengan aturan tata bahasa yang ditetapkan. Hasil dari tahap ini, berupa struktur pohon sintaksis yang stabil, akan digunakan sebagai basis utama untuk pengerjaan tahap semantic analysis dan proses kompilasi selanjutnya pada milestone berikutnya.

4.2. Saran

- 13524024 - Billie Bhaskara Wibawa

 Luigi Boccherini - Minuet - String Quintet

- 13524050 - Raysha Erviandika Putra



Tonton wistoria season 2

- 13524058 - Muhammad Naufal Romi Annafi



"welcome to dominos"

"here's your hat, here's your apron"

"get in the line"

"DOMI DOMI DOMINOS"

"and then when ye ready aw"

"tuk tuk tuk tuk... ahek.....tuk"

[peak](#)

- 13524084 - Dzaki Ahmad Al Hussainy

Menurut buku tatang sutarman



5. Lampiran

5.1. Repository

Github: [JEE-Tubes-IF2224-2026](#)

5.2. Tabel Pembagian Tugas

Nama	NIM	Workload	Persentase
Billie Bhaskara Wibawa	13524024	Laporan, Coding	25%
Raysha Erviandika Putra	13524050	Laporan, Coding	25%
Muhammad Naufal Romi Annafi	13524058	Laporan, Coding	25%
Dzaki Ahmad Al Hussainy	13524084	Laporan, Coding	25%

6. Referensi

Cooper, K. D., & Torczon, L. (2011). *Engineering a compiler* (2nd ed.). Morgan Kaufmann.

"Let's Build A Simple Interpreter. Part 7: Abstract Syntax Trees"

<https://ruslanspivak.com/lbasi-part7>

"Contoh Gambar Parse Trees"

https://textx.github.io/Arpeggio/latest/images/calc_parse_tree.dot.png